

UNIVERSIDAD DE BELGRANO

Facultad de Ingeniería y Tecnología Informática

Licenciatura en Sistemas de Información

TRABAJO FINAL DE CARRERA

**Construcción del Modelo Léxico Extendido del Lenguaje
mediante Inteligencia Artificial Generativa**

Autor: Martín Romano

Directora: Dra. Graciela D. S. Hadad

Buenos Aires, Argentina — 2026

Resumen

La construcción manual del modelo Léxico Extendido del Lenguaje (LEL) a partir de la información elicitada es una actividad central de la Ingeniería de Requisitos orientada al cliente, pero también laboriosa y propensa a omisiones e inconsistencias. Este Trabajo Final de Carrera diseña e implementa un prototipo funcional capaz de asistir en la construcción del LEL a partir de la transcripción de entrevistas grabadas, empleando Inteligencia Artificial Generativa, específicamente Grandes Modelos de Lenguaje (LLM).

El foco del trabajo es el prototipo y la evaluación de en qué medida la Inteligencia Artificial Generativa mejora la construcción del LEL frente a su construcción utilizando Procesamiento de Lenguaje Natural (PLN) tradicional. Para ponerlo a prueba se define un *Gold Standard* de referencia —un LEL construido manualmente que actúa como patrón de comparación— y se diseña la contrastación con tres enfoques: dos líneas base (*baselines*) de PLN tradicional y un pipeline basado en LLM que extrae, clasifica y describe los símbolos del LEL, con una etapa de auto-verificación. Como caso de muestreo principal se emplea ecoFactory, una empresa real cuyo LEL fue construido manualmente y publicado por los autores en el Workshop em Engenharia de Requisitos (WER 2024) a partir de entrevistas en las que los roles de distintos usuarios fueron interpretados por un experto de dicha organización, y se lo complementa con varios dominios adicionales para evaluar la generalidad del enfoque. El trabajo se apoya, además, en una línea de trabajo previa de la Universidad de Belgrano que empleó PLN para construir un bosquejo del LEL.

La evaluación combina métricas objetivas —precisión, cobertura, F1, exactitud de clasificación y calidad de las descripciones— con un análisis cualitativo de defectos, alucinaciones y manejo del lenguaje coloquial. Los resultados preliminares de los *baselines* establecen el piso contra el cual se contrasta el enfoque generativo. Se discute además las implicancias metodológicas, las amenazas a la validez y las líneas de trabajo futuro.

Palabras clave: Ingeniería de Requisitos, Léxico Extendido del Lenguaje, Inteligencia Artificial Generativa, Grandes Modelos de Lenguaje, Procesamiento de Lenguaje Natural, Elicitación.

Abstract

Manually building the Language Extended Lexicon (LEL) from elicited information is a core activity of client-oriented Requirements Engineering, yet it is laborious and prone to omissions and inconsistencies. This work designs and implements a functional prototype that assists LEL construction from the transcription of recorded interviews, using Generative Artificial Intelligence, in particular Large Language Models (LLMs).

The work extends a research line at Universidad de Belgrano that used traditional Natural Language Processing (NLP) to build a draft LEL. It relies on a real case study—the company ecoFactory— whose LEL was manually built and published by the authors at the Workshop on Requirements Engineering (WER 2024) from interviews in which the users were role-played. A reference Gold Standard is defined and three approaches are compared: two traditional NLP baselines and an LLM-based pipeline that extracts, classifies and describes the LEL symbols, with a self-verification stage. Evaluation combines objective metrics with a qualitative analysis of defects and hallucinations.

Keywords: Requirements Engineering, Language Extended Lexicon, Generative AI, Large Language Models, Natural Language Processing, Elicitation.

Índice

Resumen.....	2
Abstract.....	3
Capítulo 1 — Introducción.....	7
1.1 Contexto y motivación.....	7
1.2 Planteo del problema.....	8
1.3 Antecedentes.....	8
1.4 Preguntas de investigación.....	9
1.5 Objetivos.....	9
1.6 Hipótesis.....	10
1.7 Alcance y limitaciones.....	10
1.8 Aportes esperados.....	10
1.9 Organización del trabajo.....	10
Capítulo 2 — Marco Teórico.....	11
2.1 La Ingeniería de Requisitos.....	11
2.2 La estrategia de IR orientada al cliente.....	12
2.3 Universo de Discurso, fuentes de información y Elicitación.....	13
2.4 El Modelo Léxico Extendido del Lenguaje.....	14
2.4.1 Estructura de un símbolo.....	14
2.4.2 Tipos de símbolo.....	15
2.4.3 Principios: circularidad y vocabulario mínimo.....	16
2.4.4 Reglas para seleccionar símbolos.....	16
2.5 Proceso de construcción del LEL.....	16
2.6 Escenarios.....	17
2.7 Verificación y Validación.....	18
2.8 Gestión de Requisitos y trazabilidad.....	19
2.9 Inteligencia Artificial Generativa y Grandes Modelos de Lenguaje.....	19
2.9.1 La arquitectura Transformer.....	19
2.9.2 Pre-entrenamiento, ajuste fino y técnicas de prompting.....	20
2.9.3 Capacidades y limitaciones.....	20
2.10 Procesamiento de Lenguaje Natural tradicional.....	20
2.11 Síntesis: por qué IA Generativa para construir el LEL.....	22
Capítulo 3 — Estado del Arte.....	23
3.1 Alcance y criterio de la revisión.....	23
3.2 Automatización de la elicitación y el modelado en IR.....	23
3.3 Construcción (semi)automática de glosarios y del LEL con PLN tradicional.....	23
3.4 Grandes Modelos de Lenguaje aplicados a la IR.....	24
3.5 Entrevistas de elicitación asistidas por LLM.....	24
3.6 Síntesis comparativa.....	25
3.7 Brecha identificada y posicionamiento.....	25
Capítulo 4 — Metodología y Diseño Experimental.....	27
4.1 Encuadre metodológico.....	27
4.2 Preguntas de investigación e hipótesis.....	27
4.3 Variables del experimento.....	27
4.4 Objeto de estudio: el caso ecoFactory.....	27
4.5 Corpus de entrada.....	27
4.6 Gold Standard de referencia.....	28

4.7 Tratamientos comparados.....	28
4.8 Procedimiento experimental.....	29
4.9 Métricas.....	30
4.10 Protocolo de emparejamiento.....	30
4.11 Análisis cualitativo.....	30
4.12 Amenazas a la validez.....	30
4.13 Reproducibilidad.....	31
Capítulo 5 — Aplicación del Prototipo al Caso de Estudio ecoFactory.....	32
5.1 Presentación del caso.....	32
5.2 Actores del dominio.....	32
5.3 El circuito del pedido.....	33
5.4 La elicitación: las entrevistas.....	34
5.5 Construcción del LEL de referencia.....	34
5.6 El Gold Standard.....	34
5.7 El GS-Corpus y la brecha de recuperabilidad.....	35
5.8 Hallazgos del caso.....	36
Capítulo 6 — Diseño e Implementación del Prototipo.....	37
6.1 Visión general de la arquitectura.....	37
6.2 Estructura del proyecto y decisiones tecnológicas.....	37
6.3 Representación del LEL: el esquema de datos.....	38
6.4 El pipeline basado en LLM.....	38
6.4.1 Extracción de símbolos.....	38
6.4.2 Clasificación por tipo.....	39
6.4.3 Descripción de noción e impacto.....	39
6.4.4 Auto-verificación.....	39
6.4.5 Orquestación y configuraciones experimentales.....	40
6.5 Abstracción de proveedor de LLM.....	40
6.6 Baselines de PLN tradicional.....	40
6.6.1 Baseline de frecuencia / Pareto.....	41
6.6.2 Baseline con spaCy.....	41
6.7 El motor de evaluación.....	41
6.8 Reproducibilidad y ejecución.....	42
6.9 Comparación y justificación de las tecnologías.....	43
6.10 Obtención del corpus: transcripción de entrevistas asistida por IA.....	44
6.11 Interfaz gráfica de ejecución paso a paso.....	44
Capítulo 7 — Experimentación y Resultados.....	46
7.1 Entorno experimental y alcance de la ejecución.....	46
7.2 Datos de las entrevistas.....	46
7.3 Resultados del baseline de frecuencia (C1a).....	47
7.4 Resultados del pipeline LLM.....	48
7.4.1 Corrida de referencia.....	48
7.4.2 Resultados.....	48
7.4.3 Comparación con el PLN tradicional.....	48
7.4.4 Ejemplos del LEL generado.....	49
7.5 Síntesis de resultados obtenidos.....	50
7.6 Caso de muestreo: robustez en un dominio nuevo.....	50
7.7 Evidencia de ejecución del prototipo.....	51
7.8 Evaluación en múltiples dominios de muestreo.....	52
7.9 Estado de las corridas pendientes: C1b y la réplica ciega.....	53

Capítulo 8 — Discusión.....	55
8.1 El piso del PLN y la oportunidad del enfoque generativo.....	55
8.2 Tres capacidades distintas: identificar, clasificar, describir.....	55
8.3 El caso del «Remito»: ¿error o señal?.....	55
8.4 La contradicción de la Facturación.....	56
8.5 El rol de la revisión humana.....	56
8.6 Sobre la validez de los resultados.....	57
8.7 Un experimento aparte: entrevistas simuladas para ecoFactory.....	57
8.8 Lecciones aprendidas.....	58
Capítulo 9 — Conclusiones y Trabajos Futuros.....	59
9.1 Conclusiones.....	59
9.2 Contribuciones.....	59
9.3 Limitaciones.....	59
9.4 Trabajos futuros.....	60
9.5 Reflexión final.....	60
Bibliografía.....	61
Anexo A — Prompts del pipeline.....	64
A.1 Extracción de candidatos (01_extraccion.txt).....	64
A.2 Clasificación por tipo (02_clasificacion.txt).....	64
A.3 Descripción de noción e impacto (03_descripcion.txt).....	65
A.4 Auto-verificación (04_verificacion.txt).....	65
Anexo B — Referencia de módulos del código.....	67
Anexo C — LEL completo producido (corrida de referencia, corpus real).....	68
C.1 Símbolos de tipo Sujeto (8).....	68
C.2 Símbolos de tipo Objeto (8).....	68
C.3 Símbolos de tipo Verbo (3).....	69
Anexo D — Verificación de ejecución con modelos comerciales vía API.....	70

Capítulo 1 — Introducción

Este capítulo introduce el problema que motiva el presente trabajo, sus antecedentes, las preguntas de investigación que la guían, sus objetivos e hipótesis, su alcance y sus aportes esperados, y describe la organización del resto del documento.

1.1 Contexto y motivación

El software se construye para resolver problemas que viven en un contexto humano y organizacional. Comprender ese contexto —su vocabulario, sus reglas, sus actores y sus necesidades— es la tarea de la Ingeniería de Requisitos (IR), y es también la etapa donde se origina la mayor parte de los problemas que más tarde encarecen o hacen fracasar los proyectos [31], [32]. Dentro de la IR, la estrategia orientada al cliente propone comprender primero el lenguaje del dominio antes de definir las funciones del sistema, y para ello utiliza un modelo fundacional: el Léxico Extendido del Lenguaje (LEL) [6].

El LEL es un glosario extendido que captura el significado de los términos propios del dominio tal como los usan los clientes y usuarios, describiendo cada símbolo mediante su *noción* (qué es) y su *impacto* (cómo repercute en el contexto). Construir un buen LEL exige elicitar información —típicamente mediante entrevistas— y luego recolectar, clasificar y describir cuidadosamente cada símbolo, respetando reglas de circularidad y de vocabulario mínimo. Es una tarea intelectualmente exigente y, sobre todo, *laboriosa*: demanda tiempo experto y es propensa a omitir símbolos, a clasificarlos incorrectamente o a describirlos de manera incompleta.

En paralelo, la *Inteligencia Artificial (IA) Generativa* y, en particular, los Grandes Modelos de Lenguaje (LLM) han demostrado una capacidad notable para comprender lenguaje natural coloquial y para generar texto estructurado y contextualizado [23]. Esa capacidad abre una oportunidad concreta: asistir, mediante IA Generativa, en la construcción del LEL a partir de las entrevistas transcritas, automatizando no solo la identificación de términos —algo que el Procesamiento de Lenguaje Natural (PLN) tradicional ya hacía parcialmente— sino también su clasificación y, sobre todo, la redacción de sus descripciones.

La evidencia empírica de las últimas décadas refuerza esta prioridad: los estudios recopilados por Wiegers y Beatty [3], lo señalado por Kotonya y Sommerville [2], y por Hadad y otros [33], coinciden en que una proporción mayoritaria de los defectos que llegan a producción se originan en la etapa de requisitos, y en que el costo de corregir un defecto crece de manera no lineal con la etapa del ciclo de vida en que se detecta. Esto se pone de manifiesto en reportes recientes sobre éxitos y fracasos en proyectos de software [31], [32]. La Fig. 1.1 ilustra esta progresión: un defecto descubierto durante la operación puede costar uno o dos órdenes de magnitud más que si se hubiera detectado durante la propia elicitación.

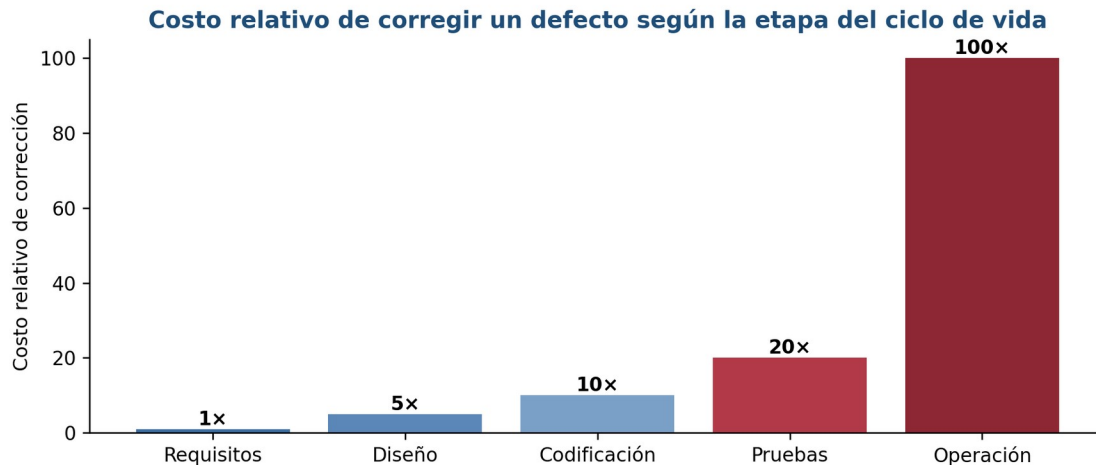


Fig. 1.1. Costo relativo de corregir un defecto según la etapa del ciclo de vida en que se detecta (valores ilustrativos de la literatura [3]).

Comprender bien el lenguaje y las necesidades del dominio antes de construir la solución es, por tanto, una de las decisiones de mayor impacto económico del ciclo de vida, y es exactamente lo que la estrategia de Ingeniería de Requisitos orientada al cliente [5] persigue con el LEL como primer modelo.

1.2 Planteo del problema

El problema que aborda este trabajo puede enunciarse así: la construcción manual del LEL a partir de la información elicitada es costosa en tiempo experto y propensa a errores de completitud y consistencia [30], y las técnicas tradicionales de PLN solo automatizan su parte más superficial —sugerir términos candidatos— sin resolver adecuadamente la clasificación semántica ni la generación de las descripciones [17], [18]. Falta, por tanto, un enfoque capaz de producir un borrador de LEL razonablemente completo y bien descrito a partir de entrevistas en lenguaje natural, reduciendo el esfuerzo manual solo a una tarea de revisión y ajuste.

Conviene precisar la naturaleza del problema. La norma ISO/IEC/IEEE 29148 [1] exige que cada requisito —y, por extensión, cada artefacto que lo sustenta, como el LEL— sea no ambiguo, completo, consistente y verificable. Un LEL incompleto o inconsistente compromete esas propiedades hacia actividades posteriores, porque, con base en la estrategia orientada al cliente [5], los escenarios y luego la especificación de requisitos se construyen sobre su vocabulario. El costo de la construcción manual y su propensión al error no son, entonces, una mera molestia operativa, sino un riesgo de calidad que se propaga a todo el proyecto.

1.3 Antecedentes

Existen dos antecedentes directos que enmarcan el trabajo. El primero es un Trabajo Final de Carrera previo de la Universidad de Belgrano [17] (y resumen publicado en [18]) que utilizó técnicas tradicionales de PLN —análisis de frecuencia, reconocimiento de entidades y análisis tipo Pareto sobre mapas conceptuales— para construir un *bosquejo* del LEL a partir de documentos descriptivos. Ese antecedente demostró el potencial de la automatización en la identificación de candidatos a símbolo, y abordó parcialmente la generación de las descripciones, mediante una heurística que asigna al símbolo las oraciones del corpus donde aparece, según su tipo. Este trabajo busca avanzar sobre esa línea con un enfoque superador para la

redacción de la noción y el impacto, apoyado en la capacidad de los LLM para generar contenido nuevo y no solo extraer fragmentos existentes del corpus.

El segundo antecedente es un trabajo previo de los propios autores de este trabajo, realizado en la asignatura Ingeniería de Software V y publicado en el *27th Workshop em Engenharia de Requisitos* (WER 2024) bajo el título «Facilitación Gráfica en Modelos de la Ingeniería de Requisitos» [13]. En ese trabajo se construyó manualmente el LEL de la empresa ecoFactory a partir de entrevistas, aplicando la técnica de facilitación gráfica [29] para mejorar la comprensión del problema. Este trabajo continúa esa línea, reemplazando la construcción manual por una asistida con IA Generativa y reutilizando el LEL ya construido como verdad de referencia, lo que constituye una base de evaluación independiente y previa al prototipo.

Ambos antecedentes comparten el método de elicitación recomendado por la estrategia y por la literatura clásica: la *entrevista*. Whitten y Bentley [4] proveen una guía estructurada para conducirlas, y Kotonya y Sommerville [2] y Wiegers y Beatty [3] documentan tanto sus virtudes —riqueza y contextualización— como sus dificultades —el carácter tácito de buena parte del conocimiento y el ruido del lenguaje conversacional—. Este trabajo hereda ese material como insumo: el *corpus* de entrada —el conjunto de entrevistas transcritas que alimenta el proceso— llega con todas sus imperfecciones, y el desafío es construir el LEL a partir de ellas de manera (semi)automática, algo que el antecedente de la UB logró sólo parcialmente y que el trabajo de WER 2024 [13] resolvió de forma enteramente manual.

1.4 Preguntas de investigación

1. **PI1.** ¿Puede la IA Generativa construir, a partir de entrevistas transcritas en lenguaje natural coloquial, un borrador de LEL con cobertura y calidad comparables a las de un LEL construido manualmente?
2. **PI2.** ¿Cómo se compara el enfoque basado en LLM con los enfoques tradicionales de PLN en identificación de símbolos, clasificación por tipo y generación de descripciones?
3. **PI3.** ¿Qué reducción de esfuerzo ofrece respecto de la construcción manual, y qué tipo de revisión humana sigue siendo necesaria?
4. **PI4.** ¿Qué clases de error introduce el enfoque generativo (alucinaciones, símbolos irrelevantes, descripciones incorrectas) y con qué frecuencia?

1.5 Objetivos

Objetivo general. Construir un prototipo funcional capaz de asistir en la construcción del modelo Léxico Extendido del Lenguaje a partir de la transcripción de entrevistas grabadas, utilizando Inteligencia Artificial Generativa.

Objetivos específicos:

- Consolidar un caso de estudio realista con entrevistas y un Gold Standard del LEL, disponible para su uso por terceros.
- Diseñar un prototipo de construcción del LEL por capas de ejecución, basado en LLM, que incluya una actividad de auto-evaluación.
- Implementar el prototipo para su uso con abstracción del modelo LLM.
- Desarrollar dos baselines de PLN tradicional a fin de compararlos con el prototipo basado en LLM.
- Definir un protocolo de evaluación de un modelo LEL con métricas objetivas y un análisis cualitativo, independiente del método de construcción.

- Evaluar el desempeño del prototipo basado en LLM frente a un Gold Standard y a baselines de PLN tradicionales.

Alcanzar estos objetivos requiere, además, un conjunto de actividades de apoyo que no constituyen objetivos en sí mismos, sino medios para lograrlos: organizar el marco teórico del modelo LEL de modo que resulte utilizable como reglas en los prompts del LLM, y analizar los trabajos relacionados con la construcción del LEL y con el uso de LLM en actividades de la Ingeniería de Requisitos.

1.6 Hipótesis

Un enfoque basado en LLM, encuadrado en el método de construcción del LEL y dotado de salvaguardas (verificación contra el corpus, control de variabilidad —reducir las diferencias entre distintas ejecuciones del modelo LLM— y auto-verificación), produce un borrador de LEL con mayor cobertura y mejor calidad de descripción que los enfoques de PLN tradicional, con una reducción sustancial del esfuerzo manual, aunque requiriendo una etapa de revisión y corrección por parte del ingeniero de requisitos.

1.7 Alcance y limitaciones

El trabajo se concentra en la primera etapa de la estrategia orientada al cliente —la construcción del LEL— y no abarca la generación automática de escenarios ni de la especificación de requisitos, que se dejan como trabajo futuro. La evaluación rigurosa se ancla en un caso de muestreo principal (ecoFactory, con corpus enteramente real) y se complementa con casos en varios dominios adicionales con entrevistas simuladas por el autor, para explorar la generalidad. Estas limitaciones se asumen explícitamente y se discuten en el capítulo de amenazas a la validez.

1.8 Aportes esperados

- Un prototipo funcional, reproducible y documentado para asistir en la construcción del LEL con IA Generativa.
- Un protocolo de evaluación determinístico y un conjunto de casos de muestreo con sus *Gold Standards* (LEL de referencia), reutilizables por la comunidad.
- Evidencia empírica comparativa entre uso de LLM y de PLN tradicional para esta tarea.
- Una discusión metodológica sobre las ventajas, los riesgos y la revisión humana necesaria del enfoque generativo en la IR.

1.9 Organización del trabajo

El Capítulo 2 presenta el marco teórico. El Capítulo 3 releva el estado del arte. El Capítulo 4 describe la metodología y el diseño experimental. El Capítulo 5 presenta el caso de estudio ecoFactory y el Gold Standard. El Capítulo 6 detalla el diseño y la implementación del prototipo. El Capítulo 7 reporta la experimentación y los resultados. El Capítulo 8 discute los hallazgos y el Capítulo 9 concluye y propone trabajos futuros.

Nota sobre reproducibilidad. El código del prototipo y de los baselines, los prompts, el corpus, el Gold Standard en formato legible por máquina, los scripts de ejecución y las salidas de las corridas se publican en un repositorio de acceso público. A lo largo del documento, las referencias a archivos o rutas concretas (por ejemplo, ``config.yaml`` o ``INSTRUCTIVO_EJECUCION.md``) remiten a ese repositorio, cuya organización se describe en el Anexo B.

Capítulo 2 — Marco Teórico

Este capítulo establece los fundamentos conceptuales sobre los que se asienta el trabajo. Recorre, en primer lugar, la disciplina de la Ingeniería de Requisitos y la estrategia orientada al cliente que enmarca el caso de estudio; luego, en profundidad, el Modelo Léxico Extendido del Lenguaje —su estructura, sus principios y su proceso de construcción—, junto con los Escenarios, la Verificación y Validación y la Gestión de Requisitos. Finalmente, presenta los fundamentos de la Inteligencia Artificial Generativa y de los Grandes Modelos de Lenguaje, así como del Procesamiento de Lenguaje Natural tradicional, para cerrar con una síntesis que justifica el enfoque adoptado.

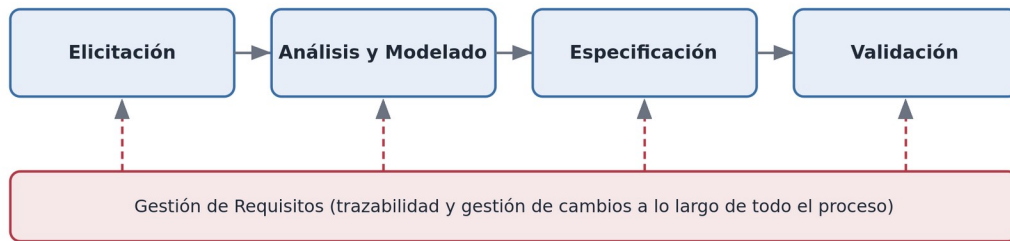
2.1 La Ingeniería de Requisitos

La Ingeniería de Requisitos es la rama de la Ingeniería de Software que se ocupa de descubrir, analizar, especificar, validar y gestionar las necesidades que un sistema de software debe satisfacer. Su objeto no es el software en sí, sino el problema que el software pretende resolver y el contexto en el que ese problema vive. Kotonya y Sommerville [2] la definen como el proceso sistemático de desarrollar requisitos a través de un proceso iterativo y cooperativo de análisis del problema, documentando las observaciones resultantes en diversos formatos de representación y verificando la exactitud de la comprensión lograda.

La importancia de la IR es difícil de sobreestimar [33]. Numerosos estudios sobre proyectos de software fallidos coinciden en que una proporción mayoritaria de las causas se origina en requisitos incompletos, ambiguos, mal entendidos o mal gestionados [31], [32]. Wiegers y Beatty [3] subrayan que el costo de corregir un error crece de manera no lineal con la etapa del ciclo de vida en que se detecta: un defecto de requisitos descubierto durante la operación del sistema puede costar uno o dos órdenes de magnitud más que si se hubiera detectado durante la propia elicitación. Por ello, invertir esfuerzo en comprender bien el problema antes de construir la solución es una de las decisiones de mayor impacto económico en todo el ciclo de vida.

La norma ISO/IEC/IEEE 29148 [1] brinda pautas para el proceso de la Ingeniería de Requisitos y, entre otras cosas, enumera las características de calidad que debe exhibir un buen requisito —necesario, no ambiguo, completo, consistente, verificable, trazable— y que un buen conjunto de requisitos debe poseer a nivel agregado. Estas características son la vara contra la cual se evalúa el producto de la IR. Aunque distintos autores proponen descomposiciones algo diferentes, existe consenso en que la IR comprende un conjunto de actividades entrelazadas, ilustradas en la Fig. 2.1.

Actividades de la Ingeniería de Requisitos



Las tres primeras producen y refinan los requisitos; la validación los contrasta con clientes y usuarios.

Fig. 2.1. Actividades de la Ingeniería de Requisitos. Las tres primeras producen y refinan los requisitos; la validación los contrasta con clientes y usuarios; la gestión de requisitos abarca todo el proceso.

- *Elicitación.* Descubrir las necesidades a partir de las fuentes de información: clientes, usuarios, documentación, sistemas existentes y el dominio. No es una recolección pasiva, porque buena parte del conocimiento relevante es tácito y debe extraerse mediante técnicas activas como entrevistas, cuestionarios, observación y talleres.
- *Análisis y modelado.* Estructurar, clasificar, priorizar y representar lo elicitado mediante modelos que faciliten la comprensión y detecten conflictos y omisiones. A partir de las necesidades elicitadas, se construyen los requisitos. El LEL y los Escenarios pertenecen a esta actividad.
- *Especificación.* Documentar de manera precisa los requisitos acordados, típicamente en una Especificación de Requisitos de Software (ERS).
- *Validación.* Confirmar con clientes y usuarios que los requisitos documentados reflejan sus necesidades; se distingue de la verificación, que controla la calidad interna de los modelos.
- *Gestión de requisitos.* Actividad transversal que mantiene la trazabilidad entre artefactos y administra los cambios a lo largo de todo el ciclo de vida.

El presente trabajo se concentra en el extremo más temprano de la cadena —la elicitación y el modelado del vocabulario del dominio— por ser el punto donde la comprensión del problema se cristaliza por primera vez y donde, en consecuencia, el impacto de la automatización es mayor.

2.2 La estrategia de IR orientada al cliente

El caso de estudio utilizado en este trabajo se construyó siguiendo la estrategia de Ingeniería de Requisitos orientada al cliente, desarrollada por Hadad [5] sobre la base de los trabajos de Leite y colaboradores. Es una estrategia dirigida por modelos en lenguaje natural y centrada en el cliente, que se distingue por construir los requisitos de manera incremental y trazable, partiendo del vocabulario del dominio antes que de las funciones del sistema. Sus rasgos característicos son: orientación al cliente, elicitación dirigida por modelos, construcción de requisitos en su contexto, énfasis en la calidad de los modelos mediante verificación y validación, y adaptabilidad a distintos modelos de proceso.

La estrategia se organiza en etapas que producen artefactos encadenados, como muestra la Fig. 2.2: comprender el vocabulario del contexto actual (que produce el LEL), comprender el contexto actual (que

produce los Escenarios Actuales), definir el contexto del software (que produce los Escenarios Futuros y el LEL del Sistema) y explicitar los requisitos de software (que produce la ERS).

Estrategia de IR orientada al cliente: artefactos y trazas

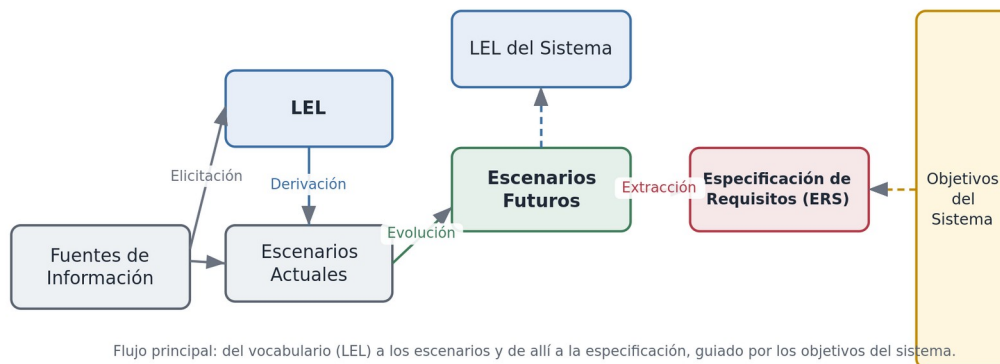


Fig. 2.2. Artefactos y trazas de la estrategia orientada al cliente. Del vocabulario (LEL) se derivan los Escenarios Actuales; estos evolucionan hacia los Escenarios Futuros —junto con el LEL del Sistema—, de los que se extrae la Especificación de Requisitos, todo guiado por los Objetivos del Sistema.

Una propiedad esencial de la estrategia es la *trazabilidad*: cada artefacto mantiene vínculos explícitos con sus fuentes y con los artefactos derivados —el LEL con las fuentes de información, los escenarios con los símbolos del LEL, los requisitos con los escenarios, y así hasta el diseño y el código—. Esta cadena de trazas permite evaluar el impacto de un cambio o auditar el origen de una decisión. El presente trabajo se sitúa en la primera etapa: la construcción del LEL.

2.3 Universo de Discurso, fuentes de información y Elicitación

La estrategia opera sobre el Universo de Discurso (UdeD): el contexto general en el que el software debe desarrollarse y operar, incluyendo todas las fuentes de información y a todas las personas relacionadas con el sistema. Las *fuentes de información* son los orígenes desde los cuales se elicitación el conocimiento: personas en sus distintos roles, documentos (formularios, comprobantes, informes, manuales, políticas), sistemas existentes y el propio entorno físico del dominio. La planificación exige identificar las fuentes, evaluarlas y seleccionar las técnicas de elicitación adecuadas a cada una.

Durante la elicitación surge a menudo información que no corresponde al vocabulario en construcción, sino a necesidades, deseos o requisitos candidatos para el sistema futuro. Esa información no se descarta: se registra en Fichas de Información Anticipada (FIA), que funcionan como un repositorio de requisitos candidatos a evaluar en etapas posteriores. Esta separación —vocabulario actual en el LEL, requisitos candidatos en las FIA— delimita qué debe y qué no debe terminar modelado como símbolo del LEL, una distinción relevante también para un método automático.

La literatura clásica de la disciplina sistematiza las *técnicas de elicitación* aplicables a esas fuentes. Kotonya y Sommerville [2] y Wiegers y Beatty [3] coinciden en un repertorio que incluye la entrevista, los *talleres* o sesiones facilitadas con varios interesados, la *observación* del trabajo real —incluida la etnografía—, los *cuestionarios*, el *análisis de documentos* y de sistemas existentes, y el uso de *escenarios* y *prototipos* como disparadores. Un punto en el que insisten estos autores es que ninguna técnica basta por sí sola: cada una ilumina un aspecto distinto del dominio, por lo que conviene combinarlas. También coinciden en la

dificultad de fondo: buena parte del conocimiento del dominio es *tácito* —los expertos lo dan por obvio y no lo verbalizan— y el ingeniero de requisitos, que no es experto en el dominio, debe hacerlo emerger.

Entre esas técnicas, la entrevista es la más utilizada y la que adopta la estrategia orientada al cliente; es, además, la fuente del corpus de este trabajo. Whitten y Bentley [4] la caracterizan en dos modalidades: la entrevista *no estructurada*, abierta y exploratoria, útil en las primeras etapas para comprender el panorama general, y la *estructurada*, guiada por un cuestionario preparado, útil para precisar detalles. Distinguen además dos tipos de pregunta cuya combinación define la riqueza del material obtenido: las *preguntas abiertas*, que invitan al entrevistado a explayarse y suelen revelar vocabulario y matices no anticipados, y las *preguntas cerradas*, que acotan la respuesta y sirven para confirmar datos puntuales. Esta distinción no es accesoria para un método automático: un corpus rico en respuestas a preguntas abiertas contiene más vocabulario de dominio —y también más ruido conversacional— que uno basado en preguntas cerradas, lo que incide directamente en lo que un LEL construido a partir de él puede capturar.

2.4 El Modelo Léxico Extendido del Lenguaje

El LEL es el modelo central del presente trabajo. Propuesto originalmente por Leite y colaboradores [6] y desarrollado extensamente por Hadad, Doorn y Kaplan [24], es un glosario extendido cuyo propósito es representar el vocabulario propio del contexto de aplicación, capturando el significado de los términos tal como los usan los clientes y usuarios, con un mínimo de interpretación por parte del ingeniero de requisitos.

El LEL persigue dos grandes objetivos. El primero es comprender el lenguaje del contexto de aplicación: asegurar una buena comunicación entre los involucrados, facilitar la validación de los modelos en lenguaje natural, preservar el mismo vocabulario a lo largo del ciclo de vida y facilitar la comprensión del UdeD. El segundo es ser un ancla para todas las fases del desarrollo: punto de partida de las actividades siguientes, repositorio consultable de conocimiento, instrumento para el entrenamiento de nuevos integrantes, fuente para la convención de nombres en el diseño y la codificación, ayuda para la documentación para usuarios e instrumento simple de trazabilidad.

2.4.1 Estructura de un símbolo

La unidad del LEL es el *símbolo*: una palabra o frase peculiar del dominio. Cada símbolo se describe mediante dos atributos complementarios, además de su nombre y sus eventuales sinónimos (Fig. 2.3). La *noción* es la descripción denotativa del símbolo: qué es, qué significa, cuál es su sentido intrínseco. El *impacto* es la descripción connotativa: cómo repercute el símbolo en el contexto, qué acciones desencadena o recibe, qué efectos produce.

Estructura del símbolo del LEL

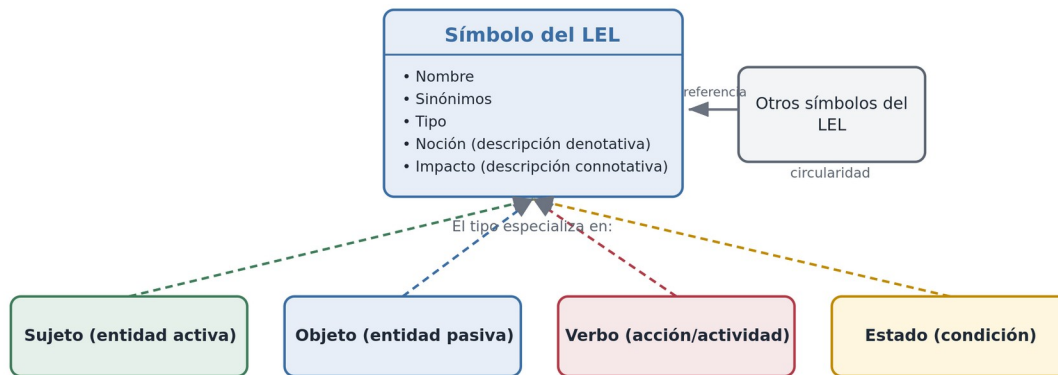


Fig. 2.3. Estructura de un símbolo del LEL. Cada símbolo posee nombre, sinónimos, tipo, noción e impacto; el tipo especializa en Sujeto, Objeto, Verbo o Estado; y las descripciones referencian otros símbolos del LEL (principio de circularidad).

2.4.2 Tipos de símbolo

Todo símbolo se clasifica en uno de cuatro tipos, según su rol en el contexto de aplicación. La Tabla 2.1 los define.

Tabla 2.1. Tipos de símbolo del LEL.

Tipo	Definición
Sujeto	Entidad activa (persona, organización, máquina o sistema) que realiza actividades en el contexto de aplicación.
Objeto	Entidad pasiva sobre la cual se aplican acciones en el contexto, sin realizar acciones por sí misma.
Verbo	Actividad o acción que ocurre en el contexto de aplicación.
Estado	Condición o situación en la que se encuentran sujetos, objetos o actividades en un momento dado, y que puede cambiar a otra condición.

El tipo determina qué debe contener la noción y el impacto, mediante plantillas de descripción (Tabla 2.2). Estas plantillas son decisivas para este trabajo, porque constituyen el criterio formal de calidad de las descripciones que un método automático debe producir.

Tabla 2.2. Plantillas de descripción según el tipo de símbolo.

Tipo	Noción	Impacto
Sujeto	Quién es, por su rol, posición o responsabilidad.	Las actividades que realiza.
Objeto	Qué representa, sus características y su relación con otros objetos.	Las acciones que se le aplican o que se realizan con él.

Tipo	Noción	Impacto
Verbo	El proceso o actividad que representa, mediante su propósito; quién lo ejecuta, cuándo y dónde.	Acciones y procedimientos involucrados; situaciones que lo impiden y otras que desencadena.
Estado	Qué representa y qué estados o actividades condujeron a él.	Otros estados y actividades que pueden ocurrir a partir de él.

Dos precisiones del método, relevantes para clasificar correctamente los símbolos, merecen destacarse. Primero, los símbolos de tipo Estado son poco frecuentes: en el vocabulario de los usuarios, los calificadores —por ejemplo, *moroso*, *anulado* o *en curso*— suelen reemplazar a los estados, de modo que ni el ingeniero ni un método automático deben crear estados que no existan realmente en el Universo de Discurso; un calificador solo se registra como Estado cuando el contexto lo utiliza como una abstracción propia. Segundo, los símbolos de tipo Verbo pueden aparecer en forma verbal —por ejemplo *Facturar*— o nominal —*Facturación*—: en este último caso hay que discernir si el sustantivo denota la actividad (Verbo) o su resultado (Objeto), o si el contexto lo usa con ambos significados. Por convención, los símbolos Verbo se nombran en *infinitivo* y en *voz activa*. Estas distinciones son, precisamente, las que el prototipo debe resolver en su etapa de clasificación.

2.4.3 Principios: circularidad y vocabulario mínimo

Dos principios gobiernan la redacción del LEL y le dan su carácter de red semántica autocontenida. El *principio de circularidad* (o de cierre) establece que, al describir un símbolo, deben utilizarse tantos otros símbolos del LEL como sea posible; así, las descripciones remiten unas a otras y el glosario se vuelve un grafo de significados interconectados. El principio de vocabulario mínimo establece que debe minimizarse el uso de términos ajenos al LEL —vocabulario externo o de dominio público— en las descripciones. Para un método automático, respetar estos principios es un desafío particular: no basta con describir cada símbolo aisladamente, sino que hay que hacerlo en función de los demás.

2.4.4 Reglas para seleccionar símbolos

No toda palabra del corpus es un símbolo. Las reglas de selección indican: seleccionar exclusivamente palabras o frases pertenecientes al contexto de la aplicación; privilegiar las frecuentemente usadas por los clientes y usuarios; identificar el nombre completo del término; y tratar las abreviaturas y acrónimos como nombre o como sinónimo según corresponda. La contracara es que los términos genéricos o de dominio público —«información», «proceso», «cosa»— no deben modelarse como símbolos. Esta frontera entre lo que pertenece al dominio y lo que es vocabulario general es una de las dificultades que un método automático debe resolver con buen criterio.

2.5 Proceso de construcción del LEL

La construcción del LEL no es un acto único, sino un proceso de cinco actividades (Fig. 2.4): tres en el flujo principal —Recolectar, Clasificar y Describir símbolos— y dos transversales de aseguramiento de calidad —Verificar y Validar— que retroalimentan el flujo.

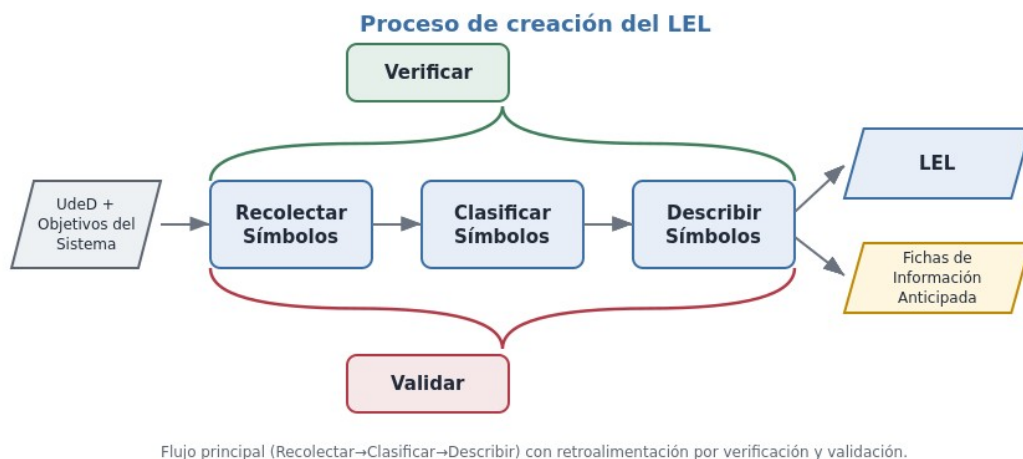


Fig. 2.4. Proceso de creación del LEL. A partir del UdeD y de los objetivos del sistema, el flujo principal recolecta, clasifica y describe los símbolos, produciendo el LEL y las Fichas de Información Anticipada; la verificación y la validación generan retroalimentaciones.

Para la actividad *recolectar símbolos* se recomiendan entrevistas no estructuradas (abiertas), haciendo preguntas solo para motivar a los clientes y usuarios a hablar, y combinándolas con la lectura de documentos. Esta recomendación es central para este trabajo, porque define la naturaleza del corpus de entrada: lenguaje natural conversacional, rico pero ruidoso. La *clasificación* asigna cada símbolo a uno de los cuatro tipos; la *descripción* redacta la noción y el impacto según la plantilla de tipos, respetando la circularidad y el vocabulario mínimo. El proceso es iterativo: la verificación y la validación detectan defectos que obligan a volver sobre las actividades del flujo principal.

La estrategia provee heurísticas concretas para cada actividad [30]. Para *clasificar*, se sugiere preguntarse si el término *ejecuta* acciones (Sujeto), *recibe* acciones sin realizarlas por sí mismo (Objeto), *nombra* una acción o procedimiento —aun nominalizado, como «Facturación»— (Verbo) o describe una *condición alcanzada* —con frecuencia un participio, como «Pedido Aprobado»— (Estado). Para *describir*, la noción responde a *qué es* el símbolo y el impacto a *cómo repercute* en el contexto, siguiendo la plantilla de su tipo; cada oración se redacta de forma atómica, se apoya en otros símbolos del LEL (circularidad) y minimiza el vocabulario externo. Estas heurísticas, propias del método, son las que el prototipo traslada de manera explícita a los prompts de cada etapa (Anexo A).

2.6 Escenarios

Los Escenarios son el segundo modelo de la estrategia orientada al cliente. Describen situaciones del dominio mediante una narración estructurada en lenguaje natural, usando el vocabulario fijado en el LEL [7]. Un escenario se compone de título, objetivo, contexto (precondición y ubicación temporal y geográfica), recursos, actores y una serie de episodios que representan la secuencia de acciones, con sus excepciones y restricciones (Fig. 2.5).

Modelo de Escenario y derivación desde el LEL

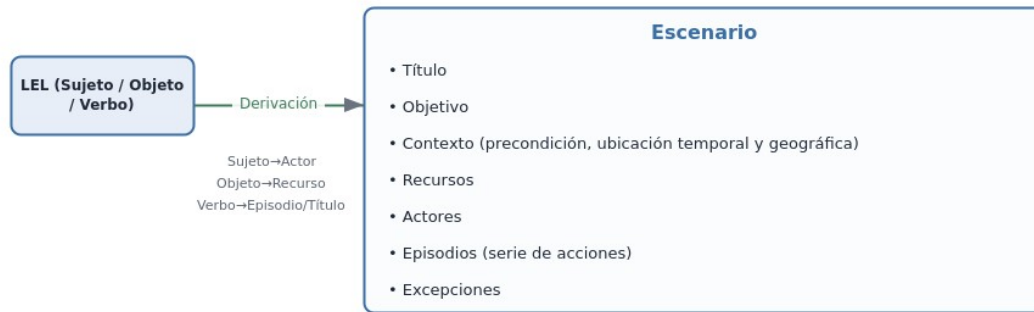


Fig. 2.5. Componentes del modelo de Escenario y su derivación desde el LEL: los símbolos de tipo Sujeto tienden a convertirse en actores, los de tipo Objeto en recursos, y los de tipo Verbo en episodios o títulos de escenario.

Los escenarios se derivan del LEL mediante una correspondencia heurística por la cual los símbolos Sujeto se vuelven actores, los Objeto se vuelven recursos y los Verbo dan lugar a episodios o a títulos de escenarios. Esto pone de manifiesto por qué la calidad del LEL condiciona toda la cadena posterior: un LEL incompleto o mal clasificado arrastra defectos hacia los escenarios y, de allí, hacia los requisitos. Aunque la generación automática de escenarios excede el alcance de este trabajo, constituye la línea natural de continuación del presente trabajo.

2.7 Verificación y Validación

La calidad de los modelos se asegura mediante *verificación* (control de la calidad interna: coherencia, completitud, cumplimiento de reglas) y *validación* (confirmación con clientes y usuarios). Para el LEL, las técnicas de verificación incluyen el análisis con checklist, las revisiones y las inspecciones. La inspección es un proceso formal y rolado (Fig. 2.6), con etapas de planificación, descripción general, preparación o lectura individual, reunión, corrección y seguimiento, con una eventual reinspección según la severidad de los defectos detectados.

Proceso de inspección (verificación)

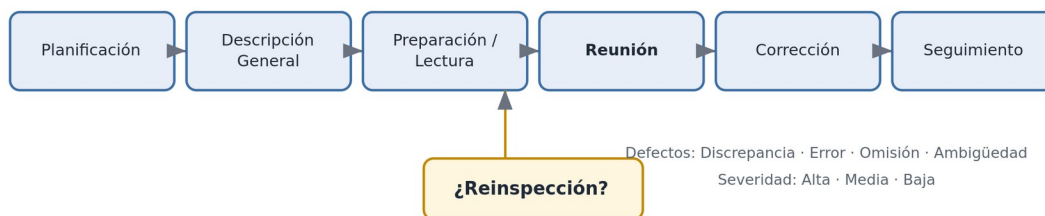


Fig. 2.6. Proceso de inspección para la verificación de los modelos, con sus etapas y la clasificación de defectos por tipo y severidad.

Durante la inspección se registran los defectos detectados, clasificados por *tipo* —Discrepancia, Error (hecho incorrecto), Omisión y Ambigüedad— y por *severidad* —Alta, Media, Baja—. Esta taxonomía es relevante por partida doble: provee el instrumento con el que se evaluará cualitativamente la calidad de los LEL generados automáticamente, e inspira la etapa de auto-verificación del prototipo, en la que el propio modelo generativo revisa su borrador contra un checklist derivado de estos criterios.

2.8 Gestión de Requisitos y trazabilidad

La *gestión de requisitos* administra la evolución de los requisitos a lo largo del tiempo. Comprende la gestión de cambios —identificar un cambio, examinar su validez, evaluar su impacto, estimar su costo y decidir su aprobación o rechazo, y luego elicitar, modificar, verificar y validar los modelos afectados— y la gestión de la trazabilidad. La trazabilidad es la capacidad de seguir la vida de un requisito hacia atrás (hasta su origen) y hacia adelante (hasta los artefactos que lo realizan) [33]. En el contexto de este trabajo incorpora de un modo concreto: para que la evaluación sea legítima, cada símbolo producido por un método automático debe poder rastrearse hasta la evidencia textual del corpus que lo sustenta. La Tabla 5.2 (Sección 5.6) documenta esa trazabilidad símbolo por símbolo para el Gold Standard de ecoFactory.

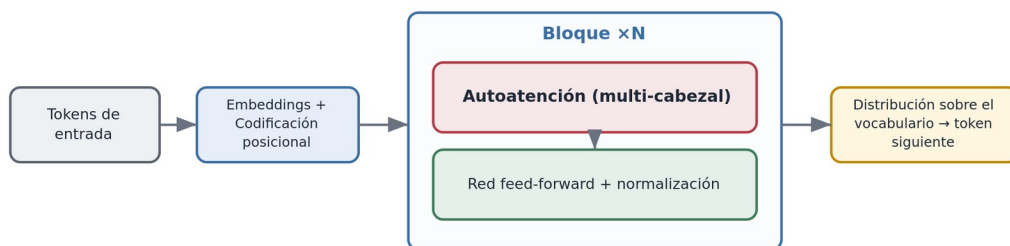
2.9 Inteligencia Artificial Generativa y Grandes Modelos de Lenguaje

La Inteligencia Artificial Generativa designa a los sistemas capaces de producir contenido nuevo —texto, imágenes, código— a partir de patrones aprendidos de grandes volúmenes de datos. En el dominio del lenguaje, su exponente actual son los Grandes Modelos de Lenguaje (LLM) [23]: redes neuronales con miles de millones de parámetros, entrenadas para modelar la probabilidad de las secuencias de texto.

2.9.1 La arquitectura Transformer

La mayoría de los LLM modernos se basan en la arquitectura *Transformer*, introducida por Vaswani et al. en 2017 [9], cuyo esquema simplificado se muestra en la Fig. 2.7. Su innovación central es el mecanismo de atención (self-attention), que permite al modelo ponderar, para cada elemento de la secuencia, la relevancia de todos los demás, sin las limitaciones de las arquitecturas recurrentes previas. El texto de entrada se convierte en tokens, estos en vectores (embeddings) a los que se añade una codificación posicional, y luego se procesan a través de una pila de bloques compuestos por autoatención multi-cabezal y una red feed-forward con normalización. A la salida, el modelo produce una distribución de probabilidad sobre el vocabulario y genera el texto de manera autorregresiva, un token por vez.

Arquitectura Transformer (esquema simplificado)



La autoatención pondera la relevancia de cada token respecto de los demás; el modelo predice el token siguiente de manera autorregresiva.

Fig. 2.7. Esquema simplificado de la arquitectura Transformer. La autoatención pondera la relevancia recíproca de los tokens; el modelo predice el token siguiente de modo autorregresivo.

Conviene detenerse en los componentes técnicos, porque condicionan cómo se diseña la interacción con el modelo. La *tokenización* parte el texto en unidades subléxicas (tokens), no siempre palabras completas, lo que explica que el modelo opere sobre fragmentos y que el costo y los límites se midan en tokens. El mecanismo de *autoatención* calcula, para cada token, una combinación ponderada de los demás a partir de tres proyecciones aprendidas —consulta, clave y valor (*query*, *key*, *value*)—, de modo que el significado de cada token se contextualiza con el resto de la secuencia; al hacerlo en varias «cabezas» en paralelo, el modelo captura distintos tipos de relación. La cantidad de tokens que el modelo puede considerar de una vez es su *ventana de contexto*, un límite práctico relevante cuando se le entregan transcripciones largas. Finalmente, la generación es *autorregresiva* y probabilística: en cada paso el modelo produce una distribución sobre el vocabulario y se muestrea el token siguiente, y parámetros como la *temperatura* —y el muestreo *top-p*— regulan cuánta aleatoriedad se introduce. Fijar una temperatura baja, como hace este trabajo, reduce la variabilidad entre corridas y favorece la reproducibilidad [23].

2.9.2 Pre-entrenamiento, ajuste fino y técnicas de prompting

Los LLM se construyen en dos grandes fases. En el *pre-entrenamiento*, el modelo aprende de forma auto-supervisada a predecir texto sobre corpus masivos, adquiriendo conocimiento lingüístico y factual de amplio espectro. En un *ajuste fino* posterior, y mediante técnicas como el aprendizaje por refuerzo con retroalimentación humana, el modelo se especializa y se alinea para seguir instrucciones. Una propiedad notable de los LLM de gran escala es el aprendizaje en contexto: la capacidad de resolver tareas nuevas a partir de las instrucciones y ejemplos provistos en la propia entrada, sin reentrenamiento.

De aquí surgen las técnicas de *prompting* que este trabajo emplea: zero-shot (se describe la tarea sin ejemplos), few-shot (se incluyen algunos ejemplos resueltos que guían el formato y el criterio esperados) y cadena de pensamiento (se induce al modelo a razonar paso a paso). El diseño del prototipo se apoya en estas técnicas y, además, descompone la construcción del LEL en etapas sucesivas (extraer, clasificar, describir, verificar), un enfoque afín a la idea de razonamiento estructurado.

2.9.3 Capacidades y limitaciones

Los LLM exhiben una notable capacidad para comprender lenguaje natural coloquial, resumir, clasificar y generar texto fluido y contextualizado, lo que los hace candidatos naturales para una tarea como la descripción de símbolos del LEL, que el PLN clásico no puede abordar apropiadamente. Sin embargo, presentan limitaciones que este trabajo considera de manera explícita. La más relevante es la *alucinación*: la tendencia a generar afirmaciones plausibles, pero no sustentadas en la evidencia. En el contexto del LEL, una alucinación se manifiesta como un símbolo inexistente en el dominio o como una oración que afirma hechos no presentes en las entrevistas. Otras limitaciones son la sensibilidad a la formulación del prompt, la variabilidad entre ejecuciones y la dificultad para garantizar una salida estrictamente estructurada. Por estas razones, el diseño experimental incorpora la verificación de alucinaciones contra el corpus, el control de la variabilidad mediante múltiples corridas y temperatura baja, y una etapa de auto-verificación.

2.10 Procesamiento de Lenguaje Natural tradicional

Antes de la irrupción de los LLM, la automatización de tareas lingüísticas se apoyaba en técnicas de Procesamiento de Lenguaje Natural (PLN) basadas en análisis estadístico y en reglas. Estas técnicas son el punto de comparación de este trabajo, ya que el antecedente académico que lo motiva empleó PLN tradicional para construir un bosquejo del LEL. Un pipeline clásico (Fig. 2.8) encadena tokenización, normalización y lematización, etiquetado gramatical (POS tagging), reconocimiento de entidades

nombradas (NER) y análisis de frecuencia —a menudo con criterios tipo Pareto— para extraer los términos candidatos.

Pipeline de PLN tradicional

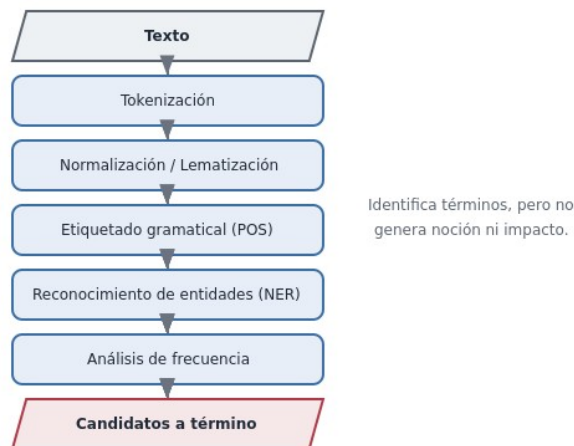


Fig. 2.8. Pipeline de PLN tradicional. Identifica términos candidatos a partir del texto, pero no genera la noción ni el impacto de los símbolos.

Herramientas como spaCy implementan estos componentes para el español con modelos estadísticos pre-entrenados. La fortaleza del PLN tradicional reside en la identificación de términos frecuentes y entidades; su limitación esencial, a los efectos del LEL, es que no comprende el significado en profundidad ni genera descripciones: puede sugerir que «factura» es un candidato, pero no puede redactar qué es una factura ni cómo impacta en el contexto, ni clasificarla semánticamente con fiabilidad. Esta asimetría de capacidades es, precisamente, el eje de la comparación experimental.

Vale precisar algunas de estas técnicas. La *frecuencia* por sí sola sobreestima los términos comunes; por eso suele ponderarse con esquemas como *TF-IDF*, que realzan los términos frecuentes en un documento, pero raros en el conjunto, aproximando mejor la especificidad de un término del dominio. Una evolución posterior son los *embeddings* de palabras —word2vec, GloVe y, más tarde, los contextuales—, que representan cada término como un vector en un espacio donde la cercanía refleja similitud de uso; capturan relaciones semánticas superficiales, pero no comprenden ni redactan. El *etiquetado gramatical* y el *análisis de dependencias* aportan estructura sintáctica —útil para distinguir, por ejemplo, un sustantivo de un verbo—, y el *NER* identifica entidades como personas u organizaciones. Todas estas técnicas mejoran la *identificación* y aportan señales para una *clasificación* tentativa, pero ninguna resuelve la *descripción*: producir la noción y el impacto de un símbolo excede lo que el análisis estadístico-estructural puede ofrecer, y es justamente la brecha que el enfoque generativo aborda.

Un último elemento teórico que atraviesa este trabajo es cómo se *mide* la calidad de un LEL producido. La comparación entre enfoques se apoya en métricas heredadas de la recuperación de información: la *precisión* —qué proporción de los símbolos producidos es correcta—, la *cobertura* o *recall* —qué proporción de los símbolos esperados fue efectivamente recuperada— y su media armónica, el *F1*, que penaliza los desequilibrios entre ambas. A ellas se suma una métrica propia del problema, la exactitud de clasificación de

tipo, y una que captura la capacidad distintiva del enfoque generativo: la *cobertura de descripciones*, es decir, qué proporción de símbolos tiene noción e impacto no vacíos. Este marco de medición, desarrollado en detalle en la metodología (Capítulo 4), es el que permite contrastar de manera objetiva y reproducible el PLN tradicional con el pipeline basado en LLM.

2.11 Síntesis: por qué IA Generativa para construir el LEL

El recorrido de este capítulo permite enunciar con precisión la oportunidad que el trabajo explora. La construcción manual del LEL es valiosa pero costosa y propensa a omisiones y otros defectos [30]; el PLN tradicional automatiza solo la parte más superficial de la tarea —sugerir términos candidatos— y deja sin resolver lo esencial: clasificar los símbolos según su rol en el dominio y, sobre todo, redactar su noción y su impacto respetando las plantillas, la circularidad y el vocabulario mínimo. Los LLM, en cambio, sobresalen exactamente en lo que el PLN clásico no puede hacer —comprender lenguaje coloquial y generar texto contextualizado— aunque a costa de riesgos como la alucinación. La hipótesis del trabajo es que, encuadrado en el método de construcción del LEL y dotado de salvaguardas, un enfoque basado en IA Generativa puede producir un borrador de LEL de mayor cobertura y calidad que el PLN tradicional, con una reducción sustancial del esfuerzo manual. Los capítulos siguientes diseñan, implementan y evalúan esa hipótesis sobre el caso ecoFactory.

Capítulo 3 — Estado del Arte

Este capítulo releva los trabajos relacionados con la automatización de actividades de la Ingeniería de Requisitos, con foco en la construcción de glosarios y del LEL. Recorre primero los enfoques basados en Procesamiento de Lenguaje Natural tradicional —entre ellos el antecedente directo de la Universidad de Belgrano—, luego la aplicación emergente de los Grandes Modelos de Lenguaje a la IR, y finalmente sintetiza el panorama en una tabla comparativa para identificar la brecha que este trabajo busca cubrir.

3.1 Alcance y criterio de la revisión

La revisión se organizó alrededor de tres ejes: la automatización de la elicitación y el modelado en IR; la construcción semiautomática de glosarios y del LEL con técnicas tradicionales; y el uso de LLM en tareas de IR, en particular la extracción y el modelado de conocimiento a partir de lenguaje natural. Se priorizaron fuentes originales —actas de congresos especializados como el Workshop em Engenharia de Requisitos (WER) y repositorios reconocidos— y se contrastaron con la bibliografía de base de la disciplina.

3.2 Automatización de la elicitación y el modelado en IR

La idea de apoyar a la IR con procesamiento automático del lenguaje no es nueva. Desde hace décadas se han propuesto técnicas para extraer modelos conceptuales, identificar requisitos en documentos, detectar ambigüedades o clasificar requisitos funcionales y no funcionales a partir de texto. El supuesto común era que el lenguaje natural, pese a su ambigüedad, contiene de manera latente la estructura del dominio, y que técnicas lingüísticas y estadísticas podían recuperar parte de esa estructura. Estos trabajos lograron resultados valiosos en la identificación de términos y entidades, pero tropezaban sistemáticamente con dos límites: la comprensión semántica superficial y la incapacidad de producir descripciones en lenguaje natural de calidad comparable a la humana.

Las técnicas empleadas en esta línea provienen del Procesamiento de Lenguaje Natural clásico descrito en el marco teórico (Sección 2.10): tokenización y lematización, etiquetado gramatical (POS tagging), reconocimiento de entidades nombradas (NER) y análisis de frecuencia con criterios tipo Pareto, a menudo apoyados en herramientas como spaCy. Aplicadas a documentos de requisitos, permiten extraer términos candidatos y detectar entidades del dominio con un esfuerzo bajo, lo que explica su adopción en tareas de glosario. Sin embargo, como subrayan Kotonya y Sommerville [2] y Wieggers y Beatty [3], la información relevante de la IR es en gran medida tácita y contextual, y el análisis estadístico-superficial no la captura: *identifica qué palabras aparecen, pero no qué significan ni cómo se relacionan.*

3.3 Construcción (semi)automática de glosarios y del LEL con PLN tradicional

Dentro de esta línea se inscribe el antecedente directo de la Universidad de Belgrano [17], un Trabajo Final de Carrera que empleó técnicas tradicionales de PLN para construir un bosquejo del LEL a partir de documentos descriptivos. El enfoque combinaba análisis de frecuencia, reconocimiento de entidades y un análisis tipo Pareto sobre conceptos candidatos —apoyado en mapas conceptuales y en herramientas como las librerías NLTK y spaCy y el algoritmo TextRank— para seleccionar los candidatos a símbolo. Su aporte fue demostrar que es posible recuperar automáticamente una porción significativa de los símbolos de un LEL, acelerando la etapa de recolección, y dar un primer paso hacia la generación de descripciones mediante una heurística que asigna al símbolo las oraciones del corpus donde aparece, según su tipo. Este trabajo retoma esa línea buscando un enfoque superador para esa etapa: redactar la noción y el impacto de cada símbolo de forma más completa y consistente, apoyándose en la capacidad generativa de los LLM en lugar de una asignación posicional de oraciones.

En direcciones complementarias, Antonelli, Lezoche y Delle Ville [19] aplican PLN para *extraer conocimiento* a partir de un LEL ya construido, y Roldán Valdiviezo y Antonelli [22] proponen una gramática formal para el LEL; ya en los orígenes del modelo, Cysneiros y Leite [26] lo habían empleado para elicitar requisitos no funcionales. Todos estos trabajos confirman el interés por tratar el LEL de manera computacional, pero abordan el problema inverso —explotar un LEL existente—, su formalización o usos específicos, y no la construcción del LEL descripto a partir de entrevistas, que es el foco del presente trabajo.

Este trabajo se posiciona como una *extensión* de esa línea: conserva el objetivo (construir el LEL automáticamente a partir de fuentes en lenguaje natural) y el método de evaluación (comparación contra un LEL de referencia), pero reemplaza el PLN tradicional por IA Generativa y, sobre todo, aborda la parte que el antecedente dejaba sin resolver: la clasificación semántica y la generación de las descripciones. El propio baseline de frecuencia implementado en este trabajo reproduce, de manera controlada, el enfoque del antecedente, para que la comparación sea justa.

3.4 Grandes Modelos de Lenguaje aplicados a la IR

La aparición de los LLM reabrió el campo. Su capacidad para comprender lenguaje coloquial y generar texto estructurado los volvió candidatos para tareas que antes resultaban inabordables: clasificar requisitos, detectar inconsistencias, generar casos de uso o historias de usuario, resumir documentación y, de manera incipiente, extraer modelos conceptuales a partir de especificaciones en lenguaje natural. Trabajos recientes presentados en foros de IR exploran, por ejemplo, la extracción de un modelo conceptual desde especificaciones en lenguaje natural, una tarea estrechamente emparentada con la construcción del LEL. Sendas revisiones sistemáticas recientes [20], [25] relevan la rápida expansión de estos usos desde la irrupción de los LLM conversacionales, mientras que otros autores documentan los desafíos abiertos de aplicar LLM a tareas de IR [15]. El consenso emergente es que los LLM ofrecen un salto cualitativo en las tareas generativas, pero que requieren un encuadre metodológico cuidadoso y salvaguardas contra las alucinaciones.

Entre las aplicaciones concretas se cuentan la clasificación de requisitos funcionales y no funcionales, la detección de defectos y ambigüedades, la generación de casos de uso e historias de usuario y la extracción de modelos conceptuales —entidades y relaciones— a partir de especificaciones en lenguaje natural, esta última estrechamente emparentada con la construcción del LEL. Los fundamentos que habilitan estas capacidades son el aprendizaje en contexto y las técnicas de *prompting* —zero-shot, few-shot [10] y cadena de pensamiento [11]— descritas en la Sección 2.9. El riesgo transversal, documentado en la literatura sobre generación de lenguaje natural, es la alucinación [12]: la producción de afirmaciones plausibles, pero no sustentadas en la evidencia, que en el contexto del LEL se manifiesta como símbolos inexistentes o descripciones no respaldadas por las entrevistas. De ahí que el consenso emergente insista en un encuadre metodológico cuidadoso y en salvaguardas explícitas.

3.5 Entrevistas de elicitación asistidas por LLM

Una sublínea particularmente relevante para este trabajo es el uso de LLM en el propio proceso de entrevista y en su procesamiento posterior. Investigaciones recientes evalúan el desempeño de los LLM para pasar de las entrevistas de elicitación a los requisitos de software [14], y otros estudios analizan su uso general en la Ingeniería de Requisitos [21], retomando el clásico proceso de cuatro pasos de Christel y Kang (preparación, conducción, documentación, análisis e integración) [16]. Otros trabajos van un paso más allá y utilizan LLM para *generar los guiones* de las entrevistas de elicitación [27], o para que el modelo interprete el rol del cliente en entrevistas de práctica; esta línea ofrece, de paso, un respaldo metodológico a

la construcción de las entrevistas simuladas que emplea la presente propuesta en sus casos de muestreo adicionales (Sección 7.8). Estos trabajos confirman dos cosas útiles: que el LLM puede operar competentemente sobre el lenguaje conversacional de una entrevista, y que la calidad del resultado depende de un diseño deliberado de la interacción —lo que, trasladado a este trabajo, se traduce en el diseño cuidadoso de los prompts y de las etapas del pipeline.

El proceso de cuatro pasos de Christel y Kang [16] —preparación, conducción, documentación, y análisis e integración— sigue siendo el marco de referencia para estructurar una entrevista de elicitación, y es el que sostiene, ahora con asistencia de LLM, la evaluación del paso de entrevistas a requisitos [14], aun cuando persisten desafíos abiertos al aplicar LLM a tareas de IR [15]. Para este trabajo la lección operativa es doble: por un lado, confirma que un LLM puede procesar competentemente el discurso conversacional de una entrevista; por otro, que el resultado depende de un diseño deliberado de la interacción. Esa lección se traslada directamente al diseño de los prompts y de las cuatro etapas del pipeline (Capítulo 6), que codifican explícitamente las reglas del método de construcción del LEL en lugar de confiar en un único pedido genérico al modelo.

3.6 Síntesis comparativa

La Tabla 3.1 sintetiza el panorama y ubica el aporte de este trabajo respecto de los enfoques previos.

Tabla 3.1. Posicionamiento comparativo de los enfoques de construcción del LEL.

Enfoque	Identifica términos	Clasifica tipo	Genera noción/impacto	Evalúa vs Gold Standard	Entrada
PLN frecuencia/NER (antecedente UB) [17]	Sí	Parcial / tentativo	Muy incompleto	Parcial	Documentos descriptivos
PLN/LLM sobre el LEL [19], [22]	Sí	Sí	Parcial, según tarea	Variable	LEL / especificaciones
LLM de entrevistas a requisitos [14]	Sí	—	Parcial	Sí [14]	Entrevistas
LEL manual (estrategia orientada al cliente) [24], [5]	Sí	Sí	Sí (experto)	Referencia	Entrevistas + documentos
Este trabajo (LEL con IA Generativa)	Sí	Sí	Sí (automático + revisión)	Sí, protocolo determinístico	Entrevistas transcritas

3.7 Brecha identificada y posicionamiento

Del relevamiento de la literatura surge una brecha precisa. El PLN tradicional automatiza la identificación de términos, pero no la descripción; el LEL manual produce descripciones de calidad, pero a alto costo; y los trabajos con LLM en IR, aunque prometedores, no se han especializado en construir el LEL completo —símbolos, clasificación y descripciones— a partir de entrevistas transcritas, ni se han evaluado contra un Gold Standard del LEL con un protocolo reproducible. El presente trabajo se sitúa exactamente en esa brecha: propone, implementa y evalúa un pipeline basado en IA Generativa que produce un borrador de

LEL descripto a partir de entrevistas, y lo compara de manera rigurosa contra baselines de PLN y contra un LEL de referencia construido manualmente.

Capítulo 4 — Metodología y Diseño Experimental

4.1 Encuadre metodológico

Este Trabajo Final de Carrera adopta el paradigma de *Ciencia del Diseño* (Design Science Research, DSR), apropiado para investigaciones cuyo aporte central es la construcción y evaluación de un artefacto que resuelve un problema relevante. El artefacto es un prototipo de software que asiste en la construcción del LEL a partir de entrevistas transcritas, empleando IA Generativa. El ciclo de DSR —identificación del problema, definición de objetivos, diseño y desarrollo del artefacto, demostración, evaluación y comunicación— se corresponde con la organización de este trabajo. La fase de evaluación se instrumenta como un estudio empírico comparativo controlado contra una verdad de referencia, complementado con un análisis cualitativo; el desarrollo del prototipo sigue un ciclo de prototipado iterativo incremental.

4.2 Preguntas de investigación e hipótesis

El problema que motiva el trabajo es que la construcción manual del LEL a partir de la información elicitada es laboriosa y propensa a omisiones e inconsistencias. De allí se derivan las cuatro preguntas de investigación enunciadas en el Capítulo 1 (PI1 a PI4). La hipótesis de trabajo sostiene que un enfoque basado en LLM produce un borrador de LEL con mayor cobertura y calidad de descripción que los enfoques de PLN tradicional, con una reducción sustancial del esfuerzo manual, aunque requiriendo una etapa de revisión y corrección por parte del ingeniero de requisitos.

4.3 Variables del experimento

La *variable independiente* es el método de construcción del LEL, con sus configuraciones (sección 4.7), denominada *tratamiento*, es decir, cada enfoque automático de construcción del LEL. Las *variables dependientes* son las métricas de calidad y eficiencia (sección 4.9). Se controlan, manteniéndolos constantes para cada tratamiento, las siguientes: el corpus de entrada, el Gold Standard de referencia, el protocolo de emparejamiento y el método de construcción del LEL manual. Para el LLM se fijan, además, la versión del modelo, la temperatura y, cuando el proveedor lo permite, la semilla.

4.4 Objeto de estudio: el caso ecoFactory

El caso de estudio es *ecoFactory*, una empresa real de fabricación y distribución de bolsas ecológicas con clientes mayoristas y minoristas, depósitos en la provincia de Buenos Aires y un sistema ERP con integración a la AFIP. El caso proviene de un trabajo previo de los mismos autores, realizado en Ingeniería de Software V y publicado en WER 2024 [13], en el que se construyó manualmente el LEL de ecoFactory mediante facilitación gráfica sobre entrevistas a los roles interpretados en rol, y grabadas y transcritas. Reusar un caso y un LEL producidos de forma independiente y previa a este TFC es una decisión metodológica deliberada: el LEL de referencia no fue confeccionado a medida del prototipo, lo que elimina la contaminación entre la entrada y la salida esperada. El caso se detalla en el Capítulo 5.

4.5 Corpus de entrada

El corpus está formado por las entrevistas de elicitación transcritas del estudio original de ecoFactory, en español rioplatense coloquial. De las cinco entrevistas originales se recuperaron cuatro transcripciones reales —el Dueño y el Gerente Comercial, que capturan la visión del negocio, y el Operario del ERP y el Responsable del ERP, más enfocados en la problemática tecnológica—; de la quinta no se conserva el audio ni la transcripción. Las cuatro entrevistas reales conforman el corpus único sobre el que se ejecutan todos los

tratamientos: cada método recibe exactamente el mismo material de entrada. El diseño global se resume en la Fig. 4.1.

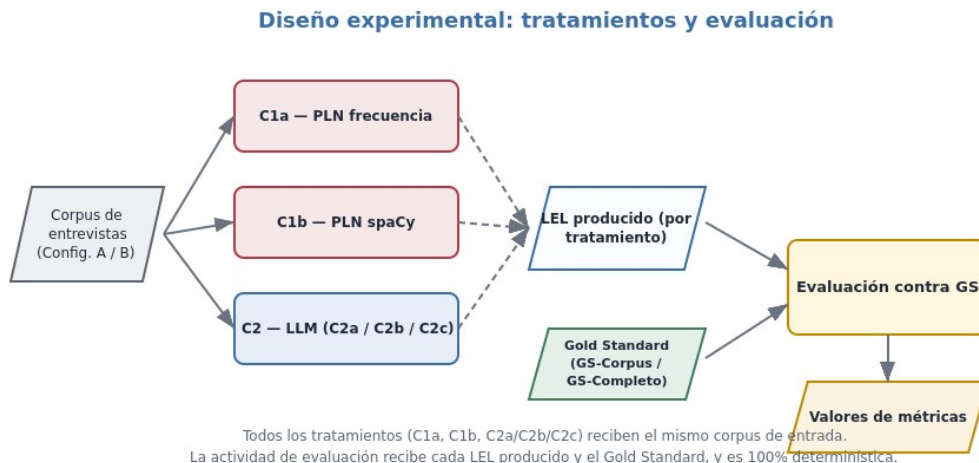


Fig. 4.1. Diseño experimental: todos los tratamientos reciben el mismo corpus de cuatro entrevistas reales y se miden contra el mismo Gold Standard mediante un protocolo de emparejamiento determinístico.

4.6 Gold Standard de referencia

La verdad de referencia es el *Gold Standard* del LEL de ecoFactory: los 21 símbolos tipados (Sujeto/Objeto/Verbo/Estado) con su noción e impacto, tal como fueron contruidos, verificados y validados manualmente en el trabajo de origen [13]. Este conjunto completo, denominado *GS-Completo*, es el LEL experto contra el que se contrasta la salida de cada método. Su tamaño —mayor que el vocabulario presente en las transcripciones— se explica porque el LEL manual no surge únicamente de las entrevistas: se fue ampliando durante el estudio original con la información anticipada, con los escenarios derivados del propio LEL y con la sesión de facilitación gráfica, todo lo cual incorpora símbolos que el vocabulario hablado en las cuatro transcripciones no contiene.

En consecuencia, no todos esos 21 símbolos tienen sustento textual en las cuatro entrevistas transcritas. Por eso se identifica, dentro del GS-Completo, el subconjunto de símbolos efectivamente *recuperables* del corpus disponible: 15 de los 21 (*GS-Corpus*). La diferencia entre ambos —la *brecha de recuperabilidad*— se analiza como un resultado en sí mismo, y no como un defecto: mide cuánto de un LEL experto puede reconstruirse a partir de transcripciones crudas. Exigirle a un método que recupere un símbolo cuya evidencia no está en el texto no sería una evaluación justa; por eso las métricas se reportan tanto contra el GS-Completo (techo teórico) como contra el GS-Corpus (blanco alcanzable).

4.7 Tratamientos comparados

Se diseñaron cinco tratamientos diferentes de modelado automático del LEL para comparar la eficacia de cada uno. Todos los tratamientos reciben el mismo corpus y se miden contra el mismo Gold Standard (Tabla 4.1).

Tabla 4.1. Tratamientos comparados en el experimento.

ID	Enfoque	Descripción
C1a	PLN — frecuencia	Réplica del antecedente UB: análisis de frecuencia/Pareto y extracción de candidatos. Identifica términos; no genera noción/impacto.
C1b	PLN — spaCy	Análisis lingüístico (POS, lematización, sintagmas nominales, NER). No genera noción/impacto.
C2a	LLM — básica	Pipeline en una pasada de extracción y clasificación, con descripción por símbolo (zero-shot), sin auto-verificación.
C2b	LLM — multi-etapa	Etapas separadas de extracción, clasificación y descripción con ejemplos (few-shot), sin auto-verificación.
C2c	LLM — completa	C2b más una etapa de auto-verificación del borrador contra el checklist de inspección.

El pipeline del prototipo LLM, con sus etapas y configuraciones, se ilustra en la Fig. 4.2. Dado que el plan establece usar «el modelo que mejor funcione», la elección del modelo LLM se resuelve empíricamente: las configuraciones C2 se ejecutan sobre dos o tres modelos líderes y se reporta cuál obtiene el mejor desempeño. La comparación entre C1 y C2 se interpreta teniendo presente que el PLN tradicional compite esencialmente en la identificación de términos, mientras que el LLM aborda además la clasificación y la descripción —tareas que el PLN clásico no puede resolver adecuadamente—, lo que constituye el aporte diferencial bajo estudio.

Pipeline del prototipo basado en LLM

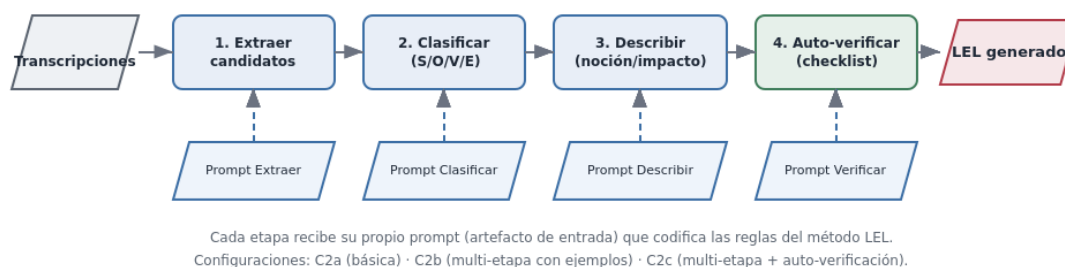


Fig. 4.2. Pipeline del prototipo basado en LLM. Cada etapa usa un prompt propio que codifica las reglas del método LEL; las configuraciones C2a, C2b y C2c activan progresivamente los ejemplos y la auto-verificación.

4.8 Procedimiento experimental

A continuación, se describen los pasos del procedimiento experimental, en el orden en que se ejecutarían:

1. Construcción del Gold Standard (tarea previa e independiente de las restantes): consolidación y verificación del LEL manual de ecoFactory, congelado como referencia.
2. Preparación del corpus: limpieza de las transcripciones y conformación de las Configuraciones A y B.
3. Ejecución de los baselines de PLN (C1a, C1b) sobre cada configuración.

4. Ejecución del pipeline LLM (C2a, C2b, C2c) sobre cada uno de los modelos LLM candidatos, con múltiples corridas por combinación para estimar variabilidad.
5. Emparejamiento de cada LEL producido contra el Gold Standard según el protocolo (sección 4.10).
6. Cómputo de métricas cuantitativas (sección 4.9).
7. Análisis cualitativo: defectos, alucinaciones y manejo del lenguaje coloquial y de las contradicciones entre fuentes.
8. Síntesis comparativa y contrastación de la hipótesis.

4.9 Métricas

Las métricas se apoyan en la literatura sobre procesamiento de la información elicitada [8] y en las métricas estándar de recuperación de información. Se calculan, cuando corresponde, contra GS-Corpus y contra GS-Completo.

- *Identificación de símbolos*: términos correctos (verdaderos positivos), omitidos (falsos negativos) e irrelevantes o inventados (falsos positivos, incluidas alucinaciones). De allí: precisión, cobertura (recall) y F1.
- *Clasificación de tipo*: exactitud de la asignación Sujeto/Objeto/Verbo/Estado sobre los símbolos correctamente identificados, con matriz de confusión.
- *Calidad de descripciones*: oraciones correctas e incorrectas en noción e impacto; completitud respecto de la plantilla; cumplimiento de las reglas del LEL (circularidad, vocabulario mínimo, atomicidad), perfilado con la taxonomía de defectos de la inspección.
- *Eficiencia*: tiempo total de construcción y tiempo promedio por símbolo, y reducción de esfuerzo frente al proceso manual; como referencia humana se dispone del tiempo de verificación del LEL manual registrado en el caso (2 h 43 min).

4.10 Protocolo de emparejamiento

El emparejamiento entre cada símbolo producido y el Gold Standard se realiza con reglas determinísticas para controlar el sesgo del evaluador: coincidencia exacta de nombre normalizado, tabla de sinónimos, contención de tokens significativos y similitud de cadena por encima de un umbral. Los símbolos producidos que resultan candidatos legítimos del dominio, aunque ausentes del Gold Standard (por ejemplo «Remito»), se adjudican aparte y se reportan en una modalidad estricta (cuentan como falso positivo) y otra adjudicada (no penalizan, e indican una posible omisión del Gold Standard).

4.11 Análisis cualitativo

Más allá de los números, se analizan el tipo y la severidad de los defectos según el checklist de inspección; la naturaleza de las alucinaciones y de los símbolos irrelevantes; el manejo del lenguaje coloquial frente al técnico; el tratamiento de las contradicciones entre fuentes (caso testigo: la facturación descrita como manual por el Operario y como casi automática por el Responsable del ERP); y el esfuerzo de revisión humana que cada enfoque demanda para alcanzar un LEL aceptable.

4.12 Amenazas a la validez

- *Validez de constructo*. ¿Las métricas capturan la calidad del LEL? Se mitiga apoyando la evaluación en la taxonomía de defectos y el checklist de inspección ya validados en la práctica.

- *Validez interna.* La subjetividad del emparejamiento se mitiga con el protocolo determinístico y una muestra de doble verificación; la contaminación de los datos de entrada se evita porque, en el caso ecoFactory, el corpus es enteramente real y el Gold Standard es previo e independiente. Persiste, en cambio, la contaminación del operador (exposición previa al Gold Standard), que solo se resuelve con la corrida ciega (Sección 7.9).
- *Validez externa.* Un único dominio principal (ecoFactory) limita la generalización; los seis casos de muestreo adicionales (Sección 7.6 y 7.8), con entrevistas simuladas por el autor, atenúan parcialmente esta amenaza, pero valen como demostración de robustez, no como evaluación rigurosa. Se reconoce como límite y se deriva a trabajos futuros.
- *Validez de conclusión.* La variabilidad del LLM se mitiga con múltiples corridas, temperatura y semilla fijas, y el reporte de la dispersión.
- Sesgo de experimentador en las entrevistas simuladas de los casos de muestreo adicionales. Se mitiga redactando las entrevistas desde el rol (no desde la lista de símbolos), incluyendo ruido y términos ajenos al Gold Standard de cada dominio. No aplica al caso ecoFactory, cuyo corpus es enteramente real.

4.13 Reproducibilidad

Se publican el código fuente del prototipo y de los baselines, los prompts exactos, el corpus, el Gold Standard en formato legible por máquina y las salidas crudas de cada corrida. Se registran la versión del modelo LLM utilizado, la temperatura y la semilla. El motor de evaluación es determinístico, de modo que cualquier tercero puede reproducir las métricas a partir de un LEL producido y el Gold Standard.

Capítulo 5 — Aplicación del Prototipo al Caso de Estudio ecoFactory

Este capítulo presenta el caso de estudio sobre el cual se evalúa el prototipo. Describe el dominio de la empresa ecoFactory, sus actores y su circuito de negocio; relata la elicitación realizada y las entrevistas que conforman el corpus; explica cómo se construyó manualmente el LEL de referencia; y detalla el Gold Standard resultante, incluyendo la distinción entre el conjunto completo y el conjunto recuperable, así como los hallazgos que el caso aporta a la discusión.

5.1 Presentación del caso

Antes de describir el caso conviene precisar su naturaleza. ecoFactory es una empresa real de fabricación y distribución de bolsas ecológicas; lo que se simuló no fue la empresa sino los usuarios entrevistados en el estudio original: los distintos roles fueron interpretados en entrevistas que se grabaron y transcribieron en la asignatura Ingeniería de Software V, y a partir de las cuales se construyó manualmente el LEL y los modelos gráficos [13]. Cuando a lo largo de este trabajo se habla de «entrevistas reales», se alude a esas cuatro transcripciones efectivamente disponibles del estudio original, por oposición a cualquier entrevista redactada por el autor de este TFC para fines de prueba (Sección 7.8). La aclaración importa para no sobreinterpretar los resultados: la empresa y su problemática son reales, pero los datos de entrada provienen de una dramatización controlada en un curso, no de un relevamiento de campo con personal efectivo de la empresa, lo que se menciona en las amenazas a la validez.

En el plano del negocio, el caso describe una empresa dedicada a la fabricación y distribución de bolsas ecológicas, con clientes que compran en volumen (*clientes mayoristas*, como cadenas de supermercados) y una operación que se apoya en un *sistema ERP* provisto por un tercero, con módulos de contabilidad, facturación y compras y ventas, y una integración con la AFIP para la emisión de comprobantes fiscales. Nota metodológica: las cuatro transcripciones reales disponibles solo dan sustento textual a una distinción parcial entre tipos de cliente —«mayorista» aparece una vez, «minorista» no aparece— por lo que los detalles más finos de segmentación comercial (condiciones de pago diferenciadas, listas de precios) que sí figuran en el Gold Standard manual probablemente provienen de la quinta entrevista no conservada o de la sesión de facilitación gráfica original, y no son verificables contra el corpus de este trabajo (ver Sección 5.8).

El caso recrea una pequeña o mediana empresa argentina en proceso de maduración de sus sistemas: conviven procedimientos automatizados por el ERP con prácticas manuales heredadas, lo que genera fricciones, errores de carga y demoras. Esta tensión entre el circuito «ideal» que el sistema soporta y el circuito «real» que los operarios ejecutan es una fuente rica de vocabulario y de situaciones, y por ello un buen banco de pruebas para evaluar la construcción del LEL.

5.2 Actores del dominio

El dominio involucra tres actores internos —el Dueño, el Gerente Comercial y el Operario del ERP— que interactúan con el Sistema ERP, y dos entidades externas a ecoFactory: el Responsable del ERP (personal de la empresa proveedora del sistema) y la AFIP. El Cliente, en particular el Cliente Mayorista, es una entidad externa adicional con la que interactúa el Gerente Comercial. La Fig. 5.1 los esquematiza.

Actores del dominio de ecoFactory



Los sujetos operan sobre el Sistema ERP; el Cliente y la AFIP son entidades externas que interactúan con la empresa.

Fig. 5.1. Actores del dominio de ecoFactory. Los sujetos internos operan sobre el Sistema ERP; el Cliente, la AFIP y el Responsable del ERP son entidades externas que interactúan con la empresa.

Cada actor aporta una perspectiva distinta del dominio: el Dueño ofrece la visión estratégica del negocio; el Gerente Comercial conoce la cartera de clientes y el circuito comercial, incluyendo las tareas manuales que hoy absorbe; el Operario del ERP conoce la operación cotidiana del sistema y sus desvíos; y el Responsable del ERP, externo a la empresa, conoce las capacidades del sistema desde el lado del proveedor (módulos, integraciones, seguridad). Esta diversidad de roles es deliberada: un LEL completo del dominio solo emerge al integrar los puntos de vista de todos ellos, y por eso la cobertura del léxico depende directamente de qué actores fueron entrevistados, como se analiza en el Capítulo 7.

5.3 El circuito del pedido

El proceso central del negocio es el circuito del pedido, ilustrado en la Fig. 5.2. Un cliente realiza un pedido; el área comercial lo somete a una aprobación que valida el crédito del cliente y la disponibilidad de stock; una vez que el pedido queda en estado aprobado, se genera la orden de entrega para la logística; tras la entrega se emite la factura, que se valida contra la AFIP; y cuando la entrega se completa con éxito, el pedido alcanza el estado finalizado.

Circuito del pedido en ecoFactory



Los recuadros ámbar son Estados del LEL (Pedido Aprobado, Pedido Finalizado); el resto, actividades y objetos del circuito.

Fig. 5.2. Circuito del pedido en ecoFactory. Los recuadros ámbar representan Estados del LEL (Pedido Aprobado, Pedido Finalizado); el resto, actividades y objetos del circuito.

Este circuito condensa buena parte de los símbolos del LEL: sujetos (Cliente, Sistema ERP, Operario), objetos (Pedido, Factura, Orden de Entrega), verbos (Agregar Pedido, Facturación) y estados (Pedido Aprobado, Pedido Finalizado). Reconstruir este circuito a partir de las entrevistas es, en buena medida, reconstruir el LEL — aunque, como se detalla en la Sección 5.6, no todos estos símbolos tienen sustento textual parejo en las cuatro entrevistas reales disponibles.

5.4 La elicitación: las entrevistas

La elicitación se realizó mediante entrevistas a los actores del dominio, transcritas y empleadas como corpus de entrada del experimento. El estudio original contempló cinco entrevistas a los distintos roles; de ellas se recuperaron cuatro transcripciones reales —el Dueño y el Gerente Comercial, en entrevistas desestructuradas, y el Operario del ERP y el Responsable del ERP, también desestructuradas—, mientras que la quinta no conserva transcripción disponible. La Tabla 5.1 resume el corpus.

Tabla 5.1. Entrevistas que conforman el corpus de ecoFactory.

#	Rol	Tipo de entrevista	Origen
1	Dueño	Desestructurada	Real
2	Gerente Comercial	Desestructurada	Real
3	Operario del ERP	Desestructurada	Real
4	Responsable del ERP	Desestructurada	Real

Las transcripciones conservan los rasgos del habla espontánea rioplatense: muletillas, redundancias, frases inconclusas y digresiones. Esta naturaleza «ruidosa» es justamente la que dificulta la extracción automática y la que permite evaluar la robustez de los distintos enfoques frente al lenguaje real, por oposición a un texto técnico y depurado.

5.5 Construcción del LEL de referencia

El LEL de referencia de ecoFactory fue construido manualmente por los autores, aplicando la estrategia orientada al cliente [5] y la técnica de facilitación gráfica [29]. El proceso siguió las actividades descriptas en el marco teórico: a partir de las entrevistas se recolectaron los símbolos candidatos, se los clasificó en Sujeto, Objeto, Verbo y Estado, y se redactaron su noción e impacto respetando los principios de circularidad y vocabulario mínimo; luego el léxico se verificó mediante inspección y se validó con la información disponible del dominio. El resultado de ese proceso —independiente y previo a este trabajo final— es el que aquí se adopta como verdad de referencia, lo que garantiza que el Gold Standard no fue confeccionado a medida del prototipo.

5.6 El Gold Standard

El Gold Standard reúne el LEL de ecoFactory en 21 *símbolos*: 6 Sujetos, 5 Objetos, 8 Verbos y 2 Estados, cada uno con su noción e impacto. Este LEL es el resultado del proceso completo del trabajo de origen: a partir de las entrevistas se recolectaron y clasificaron los símbolos, se redactaron sus descripciones, y luego el léxico se enriqueció al construir los escenarios y al incorporar información adicional del dominio, hasta alcanzar la versión que aquí se toma como verdad de referencia. La Tabla 5.2 lista los 21 símbolos e indica cuáles tienen sustento textual en las cuatro entrevistas transcritas disponibles (columna «Recuperable del corpus»).

Tabla 5.2. Los 21 símbolos del Gold Standard de ecoFactory y su recuperabilidad desde las cuatro entrevistas transcritas.

ID	Símbolo	Tipo	Recuperable del corpus
S01	Administración de Pedidos ERP	Sujeto	No
S02	Cliente Mayorista	Sujeto	Parcial
S03	Cliente Minorista	Sujeto	No
S04	Integración AFIP	Sujeto	Sí
S05	Operario	Sujeto	Sí
S06	Sistema ERP	Sujeto	Sí
O01	Bolsa Ecológica / Producto	Objeto	Sí
O02	Factura	Objeto	Sí
O03	Lista de Precios	Objeto	No
O04	Orden de Entrega	Objeto	Sí
O05	Pedido	Objeto	Sí
V01	Agregar Cliente	Verbo	Sí
V02	Agregar Factura	Verbo	Sí
V03	Agregar Pedido	Verbo	Sí
V04	Agregar Producto	Verbo	Sí
V05	Ajuste de Errores de Facturación	Verbo	No
V06	Creación de Reportes ERP	Verbo	Sí
V07	Distribución de Producto	Verbo	Sí
V08	Verificación de Facturas	Verbo	No
E01	Pedido Aprobado	Estado	No
E02	Pedido Finalizado	Estado	Sí

La etiqueta «Parcial» (Cliente Mayorista) en la Tabla 5.2 indica que el término tiene una mención textual directa en el corpus —«trabajamos solo con clientes mayoristas», en la entrevista al Gerente Comercial—, pero no la evidencia suficiente para sustentar la noción y el impacto tal como están redactados en el Gold Standard (que incluyen detalles de bonificaciones y condiciones de pago no verificables en esa entrevista). De los 21 símbolos, 15 resultan recuperables del corpus disponible; los 6 restantes provienen de la quinta entrevista no conservada o de la sesión de facilitación gráfica del estudio original.

5.7 El GS-Corpus y la brecha de recuperabilidad

Un principio metodológico atraviesa la evaluación: no es legítimo exigirle a un método que recupere símbolos cuya evidencia no está en el corpus de entrada. Por eso se distinguen dos conjuntos de referencia. El *GS-Completo* abarca los 21 símbolos del LEL manual. El *GS-Corpus* abarca los 15 símbolos que tienen sustento textual en las cuatro entrevistas transcritas. La diferencia entre ambos —los 6 símbolos restantes— constituye la *brecha de recuperabilidad*: símbolos que el LEL experto contiene pero que ningún método

podría inferir del corpus disponible, porque corresponden a información aportada por la quinta entrevista (no conservada) o por la sesión de facilitación gráfica del estudio original. La Fig. 5.3 compara la composición de ambos conjuntos por tipo de símbolo.

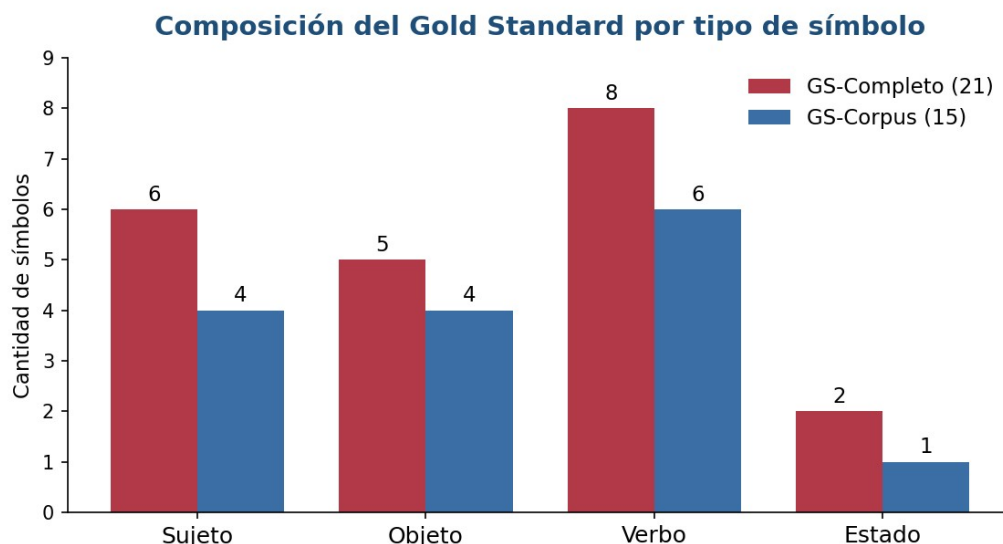


Fig. 5.3. Composición del Gold Standard por tipo de símbolo: el conjunto completo (21) frente al subconjunto recuperable del corpus (15).

Esta distinción tiene una consecuencia directa sobre el diseño experimental (Capítulo 4): cada método se evalúa contra el GS-Corpus, que es su blanco alcanzable, y también contra el GS-Completo, como techo teórico. La brecha de recuperabilidad no es, pues, un defecto, sino una variable de interés en sí misma: seis símbolos del LEL experto no pueden inferirse de las cuatro entrevistas disponibles, un dato que dimensiona hasta dónde puede llegar cualquier método —automático o manual— que parta solo de esas transcripciones (Sección 5.8).

5.8 Hallazgos del caso

La construcción del LEL y el análisis del corpus arrojaron dos hallazgos que serán retomados en la discusión. Ellos son:

1. Términos del dominio que el Gold Standard no contempla. El «Remito» aparece en las entrevistas como documento que acompaña la entrega, sin estar modelado como símbolo; un método que lo proponga no estaría necesariamente equivocado, sino señalando una posible omisión del LEL de referencia, razón por la cual el protocolo de evaluación prevé una adjudicación específica para estos casos.
2. Una contradicción entre fuentes. El Operario del ERP describe la facturación como «un proceso totalmente manual [...] se hace entrando al sistema y facturando», mientras que el Responsable del ERP —el proveedor, en una entrevista independiente— describe una gestión de facturación que genera la Factura A, B o C y permite elegir el canal de envío, un relato más cercano a lo automático. Lejos de ser un problema, esta contradicción es un fenómeno habitual de la elicitación —cada actor describe el sistema desde su rol— y un excelente caso para evaluar cómo cada enfoque maneja información en conflicto entre distintos actores.

Capítulo 6 — Diseño e Implementación del Prototipo

Este capítulo describe el artefacto construido: su arquitectura, la estructura del proyecto y las decisiones tecnológicas, la representación interna del LEL, el pipeline basado en LLM con sus cuatro etapas y prompts, la abstracción de proveedor que habilita el benchmarking de modelos LLM, los dos baselines de PLN y el motor de evaluación. Se incluyen fragmentos del código real para ilustrar las decisiones de diseño más relevantes.

6.1 Visión general de la arquitectura

El prototipo se organiza como un conjunto de módulos desacoplados que comparten una representación común del LEL (ver Anexo B). Las entradas —el corpus de entrevistas, los prompts y la configuración— alimentan tanto el pipeline basado en LLM como los baselines de PLN; ambos producen un LEL en un mismo esquema de datos; y un motor de evaluación independiente compara ese LEL contra el Gold Standard y calcula las métricas. La Fig. 6.1 resume esta organización.

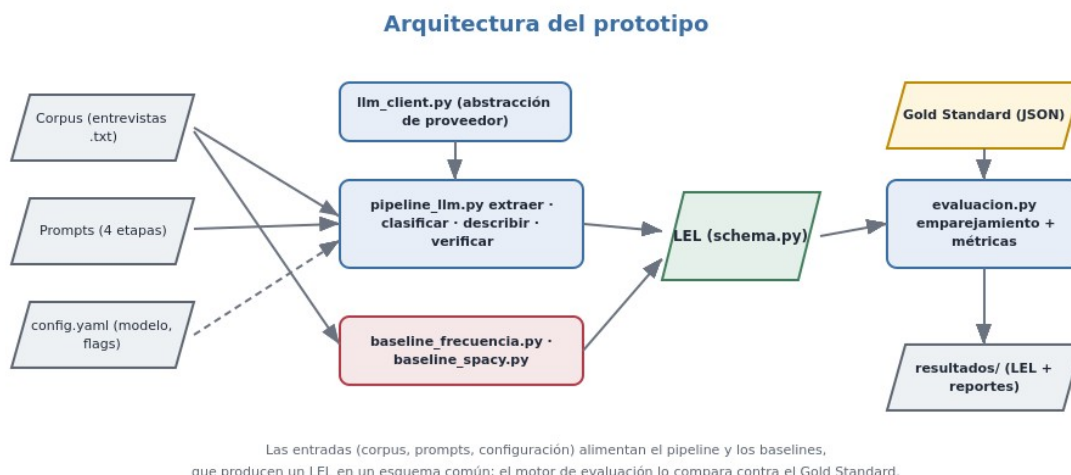


Fig. 6.1. Arquitectura del prototipo. Las entradas alimentan el pipeline y los baselines, que producen un LEL en un esquema común; el motor de evaluación lo compara contra el Gold Standard y genera los reportes.

Una decisión central de diseño es la separación estricta entre *producción* y *evaluación*. Cualquier método —LLM o PLN— produce un objeto LEL con la misma estructura, y el motor de evaluación es agnóstico respecto de cómo se generó. Esto cumple dos propósitos: hace la comparación justa (todos los métodos se miden con la misma vara y el mismo código) y mantiene la evaluación completamente determinística, sin intervención de IA, lo que la vuelve reproducible y auditable.

6.2 Estructura del proyecto y decisiones tecnológicas

El prototipo se implementó en Python. El pipeline LLM utiliza los SDK oficiales de los proveedores; los baselines usan la biblioteca estándar y, en el caso del segundo, spaCy con un modelo de español. La evaluación no requiere dependencias externas, lo que facilita su reproducción. La organización de directorios es la siguiente:

```
tfc-lel-ia/
├─ data/
│   └─ corpus/      entrevistas .txt (entrada del experimento)
```

```

├── gold/          gs_completo.json (21) y gs_corpus.json (15)
├── prompts/       01_extraccion · 02_clasificacion · 03_descripcion · 04_verificacion
├── src/
│   ├── schema.py      representación del LEL
│   ├── llm_client.py   abstracción de proveedor (OpenAI/Anthropic)
│   ├── pipeline_llm.py pipeline de 4 etapas (C2a/C2b/C2c)
│   ├── baseline_frecuencia.py PLN: frecuencia/Pareto
│   ├── baseline_spacy.py PLN: POS/NER/lematización
│   └── evaluacion.py   protocolo de emparejamiento + métricas
├── scripts/        run_baseline_frecuencia · run_baseline_spacy · run_pipeline_llm ·
run_evaluacion
├── resultados/     LEL producidos y reportes
├── webapp/         interfaz web de ejecución paso a paso (app.py + templates/)
└── config.yaml     proveedor, modelo, temperatura, corpus

```

6.3 Representación del LEL: el esquema de datos

El módulo `schema.py` define una representación única del LEL mediante clases de datos. Tener un único esquema —usado tanto por el Gold Standard como por las salidas de todos los métodos— es lo que permite que el motor de evaluación compare cualquier salida contra la referencia. Un símbolo contiene su nombre, sus sinónimos, su tipo y las listas de oraciones de noción e impacto; un LEL es una colección de símbolos.

```

@dataclass
class Simbolo:
    nombre: str
    tipo: str = "" # Sujeto | Objeto | Verbo | Estado
    nocion: List[str] = field(default_factory=list)
    impacto: List[str] = field(default_factory=list)
    sinonimos: List[str] = field(default_factory=list)
    id: str = ""

@dataclass
class LEL:
    proyecto: str = ""
    simbolos: List[Simbolo] = field(default_factory=list)

```

El módulo provee además las funciones de carga y guardado en JSON y las utilidades de normalización de nombres (minúsculas, sin acentos ni puntuación) y de extracción de tokens significativos, que el motor de evaluación reutiliza para el emparejamiento.

6.4 El pipeline basado en LLM

El corazón del prototipo es un pipeline de cuatro etapas, cada una con un prompt propio que codifica las reglas del método del LEL. Las etapas son extracción de candidatos, clasificación por tipo, descripción de noción e impacto y auto-verificación. Cada etapa construye su prompt rellenando una plantilla de texto y delega la llamada al modelo en la abstracción de proveedor (sección 6.5).

6.4.1 Extracción de símbolos

La primera etapa identifica los términos candidatos a símbolo a partir de las transcripciones, aplicando las reglas de selección del LEL. El prompt instruye al modelo a quedarse solo con vocabulario del dominio y a devolver un JSON estructurado. Un extracto de sus reglas (ver Anexo A.1):

```

Seguí estas reglas de selección de símbolos:
- Seleccioná exclusivamente palabras o frases del contexto de aplicación.

```

- Excluí palabras obvias, genéricas o de dominio público.
- Identificá el nombre completo del término.
- Una abreviatura o acrónimo puede ser nombre o sinónimo de un símbolo.

Salida: un arreglo JSON [{"nombre": "...", "sinonimos": [...]}]

6.4.2 Clasificación por tipo

La segunda etapa asigna a cada candidato uno de los cuatro tipos (Sujeto, Objeto, Verbo, Estado), proveyendo al modelo LLM las definiciones semánticas de cada tipo y ejemplos de un dominio distinto al de ecoFactory —para guiar el criterio sin contaminar el resultado—. La salida es nuevamente un JSON con la dupla nombre–tipo (ver el prompt de clasificación en el Anexo A.2).

6.4.3 Descripción de noción e impacto

La tercera etapa es la más distintiva del enfoque generativo: para cada símbolo, el modelo redacta su noción y su impacto según la plantilla de su tipo, fundamentándose únicamente en la evidencia de las transcripciones y respetando los principios de circularidad (usar otros símbolos del LEL) y de vocabulario mínimo. El prompt recibe el símbolo, su tipo, la plantilla correspondiente, la lista de los demás símbolos disponibles para referenciar y el corpus como única fuente de verdad, y exige una salida JSON con las oraciones atómicas de noción e impacto (ver Anexo A.3).

6.4.4 Auto-verificación

La cuarta etapa, opcional, somete el LEL borrador que construyó a una inspección automática: el modelo revisa cada símbolo contra un checklist derivado del proceso de inspección (adherencia a la plantilla, circularidad, vocabulario mínimo, atomicidad, ausencia de auto-referencia, ausencia de alucinaciones contra el corpus y corrección del tipo) y devuelve una versión corregida. Esta etapa traslada al propio modelo la lógica de verificación descrita en el marco teórico.

El checklist de la auto-verificación operacionaliza los criterios de las guías de inspección del LEL [5]. La Tabla 6.1 lo resume (ver el prompt en el Anexo A.4).

Tabla 6.1. Criterios del checklist de auto-verificación del pipeline.

Criterio	Qué controla
Adherencia a la plantilla	Que la noción y el impacto contengan lo que prescribe el tipo del símbolo.
Circularidad	Que las descripciones referencien, cuando sea posible, otros símbolos del LEL.
Vocabulario mínimo	Que se minimice el uso de términos ajenos al LEL.
Atomicidad	Que cada oración exprese una sola idea y tenga un solo verbo principal.
Vigencia de la información	Que cada oración distinga lo que efectivamente ocurre («es/hace») de lo que no ocurre pero debiera ocurrir («debiera ser/hacer»).
Sin auto-referencia	Que un símbolo no se describa empleándose a sí mismo.
Sin alucinaciones	Que toda afirmación tenga respaldo en la evidencia del corpus.
Corrección del tipo	Que el tipo asignado (Sujeto/Objeto/Verbo/Estado) sea el adecuado.

6.4.5 Orquestación y configuraciones experimentales

La función `construir_lsl` orquesta las cuatro etapas y, mediante banderas de configuración, materializa los tratamientos C2a, C2b y C2c definidos en el diseño experimental. El nombre del proyecto es un parámetro más —no una constante— para que la misma función sirva para cualquier caso, incluidos los de muestreo del Capítulo 7:

```
def construir_lsl(corpus_paths, llm_cfg, pipe_cfg, proyecto):
    cli = get_client(llm_cfg)
    corpus = cargar_corpus(corpus_paths)
    candidatos = extraer_candidatos(corpus, cli)          # etapa 1
    tipos = clasificar(candidatos, corpus, cli)          # etapa 2
    simbolos = []
    for c in candidatos:
        nocion, impacto = [], []
        if pipe_cfg.descripcion_por_simbolo and tipos[c["nombre"]]:
            nocion, impacto = describir(c["nombre"], tipos[c["nombre"]],
                                       otros, corpus, cli) # etapa 3
        simbolos.append(Simbolo(nombre=c["nombre"], tipo=tipos[c["nombre"]],
                                nocion=nocion, impacto=impacto))
    lsl = LSL(proyecto=proyecto, simbolos=simbolos)
    if pipe_cfg.auto_verificacion:
        lsl = auto_verificar(lsl, corpus, cli)            # etapa 4
    return lsl

CONFIGURACIONES = {
    "C2a": PipelineConfig(few_shot=False, auto_verificacion=False),
    "C2b": PipelineConfig(few_shot=True, auto_verificacion=False),
    "C2c": PipelineConfig(few_shot=True, auto_verificacion=True),
}
```

6.5 Abstracción de proveedor de LLM

Para poder ejecutar el mismo pipeline contra distintos modelos —y así resolver empíricamente cuál funciona mejor— el módulo `llm_client.py` define una interfaz común y selecciona la implementación concreta según la configuración. Las claves de API se leen de variables de entorno y nunca se escriben en el código. Dos proveedores operan sin acceso a la red: `echo`, que valida el armado del cliente, y `mock`, que reproduce offline el LSL de la corrida de referencia para demostrar el flujo completo (Sección 6.8).

```
def get_client(cfg):
    p = cfg.proveedor.lower()
    if p == "openai": return OpenAIClient(cfg)
    if p == "anthropic": return AnthropicClient(cfg)
    if p == "echo": return EchoClient(cfg) # valida el armado del cliente
    if p == "mock": return MockClient(cfg) # offline, reproduce la corrida de referencia
```

Cambiar de modelo o de proveedor para el benchmarking se reduce, entonces, a editar `config.yaml`, sin tocar el código del pipeline.

6.6 Baselines de PLN tradicional

Para que la comparación sea significativa, el prototipo incluye dos baselines de PLN que representan el estado de la técnica previo a los LLM. Ambos identifican términos candidatos, pero dejan vacías la noción y el impacto, evidenciando la limitación que el enfoque generativo busca superar.

6.6.1 Baseline de frecuencia / Pareto

Este baseline reproduce la parte por frecuencia del antecedente con PLN [17]: tokeniza el corpus, extrae n-gramas (de una a tres palabras) que no empiezan ni terminan en palabras vacías, los cuenta y aplica un corte tipo Pareto sobre la frecuencia acumulada, asignando un tipo tentativo mediante heurísticas superficiales. Cabe aclarar que aquel antecedente empleaba, además de la frecuencia, otras herramientas de PLN —las librerías NLTK y spaCy y el algoritmo TextRank— con las que llegaba a esbozar noción e impacto, aunque de manera muy pobre; aquí se aísla deliberadamente el componente por frecuencia para tener un baseline simple y controlado, y el análisis con spaCy se trata por separado como el tratamiento C1b. El núcleo del corte es:

```
cand = [(t, f) for t, f in frec.items()
        if f >= min_frec and normalizar(t) not in RUIDO]
cand.sort(key=lambda x: (-x[1], x[0]))
# corte tipo Pareto sobre la frecuencia acumulada
total = sum(f for _, f in cand) or 1
acum, seleccion = 0, []
for t, f in cand:
    seleccion.append((t, f)); acum += f
    if acum / total >= pareto or len(seleccion) >= tope:
        break
```

6.6.2 Baseline con spaCy

El segundo baseline aplica análisis lingüístico real con spaCy y un modelo de español: extrae sintagmas nominales y entidades nombradas como candidatos (clasificando las entidades de persona u organización como Sujeto), toma los lemas de los verbos principales como Verbo y los participios atributivos como Estado. Es más sofisticado que el de frecuencia, pero comparte su límite esencial: no genera descripciones.

```
nlp = spacy.load("es_core_news_md")
doc = nlp(texto)
candidatos = set()
for ent in doc.ents:
    # entidades nombradas -> Sujeto
    if ent.label_ in ("PER", "ORG"):
        candidatos.add((ent.text, "Sujeto"))
for chunk in doc.noun_chunks:
    # sintagmas nominales -> Objeto
    candidatos.add((chunk.root.lemma_, "Objeto"))
for tok in doc:
    # verbos principales -> Verbo
    if tok.pos_ == "VERB":
        candidatos.add((tok.lemma_, "Verbo"))
```

6.7 El motor de evaluación

El módulo `evaluacion.py` implementa el protocolo de emparejamiento (Fig. 6.2) y el cálculo de métricas descriptos en la metodología. Para cada símbolo producido intenta, en orden, una coincidencia exacta de nombre normalizado, una coincidencia por la tabla de sinónimos y una coincidencia por contención de tokens o similitud de cadena por encima de un umbral; los símbolos producidos sin emparejar son falsos positivos y los símbolos del Gold Standard no cubiertos son falsos negativos.

```
def _match(prod, gold):
    a, b = norm(prod.nombre), norm(gold.nombre)
    if a == b:
        # 1) nombre normalizado
        return True
    if a in sinonimos(gold) or b in sinonimos(prod):
```

```

    return True # 2) tabla de sinónimos
    ta, tb = tokens(a), tokens(b) # 3) contención de tokens
    if ta and (ta <= tb or tb <= ta):
        return True
    return SequenceMatcher(None, a, b).ratio() >= 0.85 # 4) similitud

```

Protocolo de emparejamiento y métricas

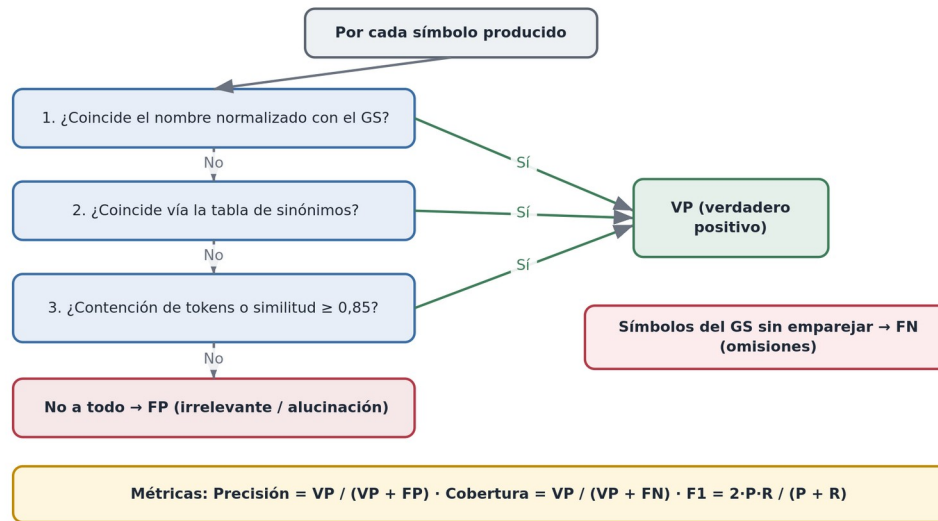


Fig. 6.2. Protocolo de emparejamiento determinístico entre los símbolos producidos y el Gold Standard, y las métricas derivadas del recuento de verdaderos positivos, falsos positivos y falsos negativos.

A partir de ese recuento se calculan las métricas estándar de recuperación de información:

```

precision = vp / (vp + fp) if (vp + fp) else 0.0
cobertura = vp / (vp + fn) if (vp + fn) else 0.0
f1 = 2 * precision * cobertura / (precision + cobertura) if (precision + cobertura) else 0.0

```

El motor calcula además la exactitud de clasificación de tipo sobre los símbolos correctamente identificados, junto con su matriz de confusión, y emite un reporte tanto en estructura de datos como en formato legible. Por ser completamente determinístico, garantiza que dos evaluaciones del mismo LEL produzcan idénticos resultados.

6.8 Reproducibilidad y ejecución

Cada método se ejecuta mediante un script dedicado que produce un LEL en el directorio resultados/, y la evaluación se lanza sobre cualquier LEL producido contra ambos conjuntos de referencia. La configuración del modelo, la temperatura y el corpus se centralizan en config.yaml; los prompts viven como archivos de texto editables; y el Gold Standard se versiona en formato JSON. De este modo, un tercero puede reproducir tanto la generación —con su propia clave de API— como, sobre todo, la evaluación, que no depende de ningún servicio externo.

Para poder validar el pipeline sin acceso a la red, la abstracción de proveedor incluye, además de OpenAI y Anthropic, un proveedor *mock* que reproduce, sin llamar a ningún modelo, el LEL de la corrida de referencia: sirve, etapa por etapa, sus símbolos reales con su tipo, noción e impacto. Esto permite ejecutar el pipeline de punta a punta de forma offline y comprobar que la orquestación, el relleno de prompts, el parseo

y el armado del esquema funcionan. Debe tenerse presente que su salida es un resultado *pre-cargado*, no una inferencia en vivo, y así corresponde presentarlo. La evidencia de estas ejecuciones se documenta en el Capítulo 7 (Sección 7.7).

6.9 Comparación y justificación de las tecnologías

El diseño del prototipo involucra varias decisiones tecnológicas que conviene justificar y situar frente a sus alternativas.

Lenguaje y ecosistema. Se eligió *Python* por ser el ecosistema de referencia para el Procesamiento de Lenguaje Natural y la interacción con LLM: dispone de las bibliotecas de PLN (spaCy), de los SDK oficiales de los proveedores de modelos y de un manejo natural de datos y de JSON, con un código legible que facilita la reproducibilidad. Alternativas como Java o JavaScript son viables, pero ofrecen un soporte menos maduro para estas tareas.

Modelo generativo. En lugar de comprometerse con un único modelo, el prototipo define una *abstracción de proveedor* que permite intercambiarlos y compararlos empíricamente (Sección 6.5). Los candidatos plausibles para la tarea, con sus compromisos, se resumen en la Tabla 6.2. La decisión de *cuál* usar no se toma a priori, sino que se resuelve con la evaluación; la arquitectura garantiza que cambiar de modelo no exija reescribir el pipeline.

Tabla 6.2. Modelos generativos candidatos para el pipeline y sus compromisos.

Modelo / familia	Proveedor	Fortalezas	Consideraciones
GPT-4 / GPT-4o	OpenAI	Alta capacidad general, buen desempeño en español, salida estructurada (modo JSON).	API paga; los datos se procesan en un tercero.
Claude	Anthropic	Ventana de contexto amplia, buen seguimiento de instrucciones complejas.	API paga; misma consideración de privacidad.
Gemini	Google	Integración con el ecosistema Google, capacidades multimodales.	API paga; misma consideración de privacidad.
Llama / Mistral (abiertos)	Meta / Mistral	Ejecución local (privacidad de los datos), sin costo por token.	Requieren hardware propio; calidad algo menor en español.

Prompting frente a ajuste fino y agentes. El pipeline se apoya en *prompting* estructurado —instrucciones, ejemplos (*few-shot*) y cadena de pensamiento— y no en ajuste fino (*fine-tuning*), que exigiría un conjunto de datos etiquetado del que no se dispone y un costo mayor, sin garantía de superar al prompting bien diseñado para esta tarea. Una alternativa emergente son los *agentes*: sistemas en los que el LLM planifica y ejecuta varios pasos de forma autónoma, invocando herramientas. Para la construcción del LEL, un enfoque agéntico podría, por ejemplo, decidir por sí mismo cuándo releer el corpus o cuándo pedir una aclaración; y, en el propio proceso de elicitación, un *agente conversacional* podría conducir la entrevista, una línea ya explorada en la literatura reciente [14], [27]. Este trabajo adopta el enfoque de etapas fijas por ser más transparente, controlable y evaluable, y deja el enfoque agéntico como trabajo futuro.

Herramientas de desarrollo. El proyecto se versiona con *Git* y *GitHub* y se desarrolla en *VS Code*. Para acelerar la escritura y la depuración del código, es razonable apoyarse en asistentes de programación basados en IA integrados al editor —como GitHub Copilot o el chat de Copilot de VS Code—, que aplican al *desarrollo* la misma clase de tecnología que el prototipo aplica al *dominio*. La evaluación, en cambio, se implementó como un motor determinístico sin dependencias externas, una decisión deliberada: garantiza que los resultados sean reproducibles por cualquier tercero sin claves ni servicios de por medio, lo que es esencial para la validez del experimento.

6.10 Obtención del corpus: transcripción de entrevistas asistida por IA

El corpus de entrada de este trabajo son entrevistas ya transcriptas, pero conviene señalar que la propia transcripción —el paso previo— también puede automatizarse con IA, completando así una visión de flujo asistido de punta a punta. Tradicionalmente, transcribir una entrevista grabada es una tarea manual, lenta y propensa a errores. Los modelos actuales de reconocimiento automático del habla (ASR), como *Whisper* de OpenAI [28], transcriben audio a texto con alta exactitud, incluso en español y en condiciones de audio no ideales, y admiten la *diarización* —la separación de los turnos de cada interlocutor—, que es precisamente el formato por hablante que el pipeline espera.

Incorporar la transcripción por IA transforma el flujo de la elicitación en: grabar la entrevista → transcribir con IA → construir el LEL con IA (este prototipo) → revisión humana. Las salvedades son análogas a las del resto del trabajo: los términos propios del dominio pueden transcribirse mal y requieren una corrección posterior, la calidad del audio condiciona el resultado, y el resguardo de la privacidad de las grabaciones es una responsabilidad ineludible. Con esos recaudos, la transcripción asistida por IA es un complemento natural de este trabajo y refuerza la hipótesis de fondo: que la IA Generativa puede reducir sustancialmente el esfuerzo manual a lo largo de todo el proceso de Ingeniería de Requisitos, dejando al ingeniero el rol de revisión y decisión.

6.11 Interfaz gráfica de ejecución paso a paso

Para hacer tangible el funcionamiento del pipeline y facilitar su demostración, se implementó una pequeña interfaz gráfica web que ejecuta el prototipo etapa por etapa. La interfaz no incorpora dependencias externas —se sirve con la biblioteca estándar de Python, mediante el módulo *http.server*— y reutiliza exactamente el mismo motor que la línea de comandos: no reimplementa lógica alguna, sino que orquesta las mismas funciones de extracción, clasificación, descripción, auto-verificación y evaluación definidas en *src/*. De este modo, lo que la interfaz muestra en pantalla es fiel a lo que produce la ejecución por consola.

El usuario dispone de un botón por cada etapa del pipeline —extraer candidatos, clasificar, describir y auto-verificar—, que se habilitan de manera secuencial: cada etapa toma como entrada el resultado de la anterior y despliega un resumen de su salida (los términos hallados, su clasificación por tipo, las descripciones de noción e impacto generadas y el conteo final). Un panel de configuración permite elegir el proveedor de LLM —incluido el proveedor *mock*, que ejecuta el flujo completo sin conexión— y, de forma opcional, el modelo, en línea con la abstracción de proveedor descrita en la Sección 6.5. La última acción genera los dos reportes del trabajo —el LEL navegable y la evaluación contra el Gold Standard— y ofrece los enlaces para abrirlos.

La interfaz permite además elegir el *corpus* sobre el que se ejecuta —el de *ecoFactory* o cualquiera de los casos de muestreo, declarados en la configuración—, evaluando cada uno contra su propio LEL de referencia. En cuanto al diseño visual, la interfaz adopta la identidad de la Universidad de Belgrano y reutiliza el mismo sistema de tarjetas, los colores por tipo de símbolo y la tipografía del reporte HTML del

LEL, de modo que la herramienta de ejecución y sus reportes conforman un conjunto visual coherente. La Fig. 6.3 muestra la interfaz tras una corrida completa con el proveedor *mock*, que reproduce offline el LEL de la corrida de referencia (un resultado pre-cargado, no una inferencia en vivo). El código se publica en el directorio *webapp/* del repositorio (Anexo B).

Universidad de Belgrano — Construcción del LEL con IA Generativa

Trabajo Final de Carrera — Autor: Martín Romano

UB

Configuración

PROVEEDOR

Mock — offline, sin API

MODELO (OPCIONAL)

p. ej. el modelo vigente

CORPUS

ecoFactory — entrevistas reales (WER 2024)

Cargar Corpus

Reiniciar

Proveedor: mock · Modelo: n/a · 4 entrevistas · 20,557 caracteres

entrevista_1_dueno.txt · 8,158 car.

entrevista_2_gerente_comercial.txt · 4,952 car.

entrevista_3_operario.txt · 5,046 car.

entrevista_4_responsable_erp.txt · 2,401 car.

1

Extraer Candidatos

Identifica los términos propios del dominio en las entrevistas.

Ejecutar

Se extrajeron 19 términos candidatos del dominio.

Dueño

Gerente Comercial

Operario ERP

Responsable ERP

Sistema ERP

Cliente

Cliente Mayorista

AFIP

Bolsa Ecológica

Pedido

Remito

Factura

Depósito

Stock

Base de Datos

Backup

Facturación

Automatizar

Integración

2

Clasificar Símbolos

Asigna a cada término un tipo: Sujeto, Objeto, Verbo o Estado.

Ejecutar

Fig. 6.3. Interfaz de ejecución paso a paso del prototipo. Un botón por etapa (extraer, clasificar, describir, auto-verificar) muestra el resultado de cada paso; la acción final genera el reporte del LEL y el de evaluación. Un selector permite elegir el corpus (ecoFactory o los casos de muestreo). Reutiliza el mismo motor que la línea de comandos y la identidad visual del reporte.

Capítulo 7 — Experimentación y Resultados

Este capítulo reporta la experimentación realizada y los resultados obtenidos. Siguiendo el diseño definido en el Capítulo 4, se ejecutan los tratamientos sobre el corpus de ecoFactory y se los mide contra el Gold Standard correspondiente. Se presentan en detalle los resultados del baseline de frecuencia (C1a), que establecen el piso de referencia, y los de la corrida de referencia del pipeline basado en LLM (C2c); C1b y las corridas ciegas con modelos comerciales quedan como corrida pendiente, explicitada en la Sección 7.9.

7.1 Entorno experimental y alcance de la ejecución

Los experimentos se organizaron en torno a los tratamientos C1a (PLN por frecuencia), C1b (PLN con spaCy) y C2a/C2b/C2c (pipeline LLM), para ser evaluados con base en el corpus de cuatro entrevistas reales. Sin embargo, no todos estos tratamientos pudieron llevarse a la práctica.

El baseline de frecuencia (C1a) y el motor de evaluación no requieren dependencias externas ni acceso a la red, por lo que se ejecutaron de manera completa y sus resultados se reportan a continuación. El baseline con spaCy (C1b) requiere el modelo de lenguaje de español, y el pipeline LLM con modelos comerciales requiere acceso a la API del proveedor; ninguno de estos se ejecutó como parte de la evaluación de este capítulo, por lo que quedan como corridas pendientes, formalizadas en la Sección 7.9 y en el `INSTRUCTIVO_EJECUCION.md` del repositorio; con posterioridad se verificó, de manera operativa, que el pipeline se ejecuta correctamente contra la API de un proveedor comercial (Anexo D), sin que ello altere los resultados que aquí se reportan. En consecuencia, este capítulo presenta los resultados del tratamiento C1a —que fijan el piso cuantitativo del problema— y una corrida de referencia del pipeline LLM (C2c) con el asistente como modelo, dejando explícitamente pendientes C1b y las corridas ciegas con modelos comerciales.

7.2 Datos de las entrevistas

El corpus de entrada está conformado por las cuatro transcripciones reales disponibles del estudio original: el Dueño, el Gerente Comercial, un Operario del ERP y el Responsable del ERP (el proveedor del sistema). La Tabla 7.1 resume sus características y su extensión en palabras.

Tabla 7.1. Datos de las entrevistas del corpus.

Entrevista	Tipo	Origen	Palabras
1. Dueño	Desestructurada	Real	1304
2. Gerente Comercial	Desestructurada	Real	759
3. Operario ERP	Desestructurada	Real	830
4. Responsable ERP	Desestructurada	Real	346

Las cuatro entrevistas conforman el corpus único sobre el que se ejecutan todos los tratamientos. Los símbolos de tipo Sujeto provienen sobre todo de la entrevista al Dueño y al Gerente Comercial; los de tipo Verbo y Objeto, en su mayoría, de las entrevistas al Operario y al Responsable del ERP, más enfocadas en la operación del sistema.

7.3 Resultados del baseline de frecuencia (C1a)

El baseline de frecuencia produce un LEL de 40 *símbolos candidatos* a partir del corpus (sin noción ni impacto). Se lo evalúa contra el *GS-Corpus* (15 símbolos, los que tienen sustento textual en las cuatro entrevistas) y también contra el *GS-Completo* (21), como techo teórico. Las métricas reportadas son verdaderos positivos (VP), falsos positivos (FP), falsos negativos (FN), precisión, cobertura, F1, exactitud de clasificación de tipo (sobre los VP) y cobertura de descripciones (Descr.), en la Tabla 7.2.

Tabla 7.2. Baseline de frecuencia (C1a) sobre el corpus de cuatro entrevistas reales.

Blanco de comparación	VP	FP	FN	Precisión	Cobertura	F1	Tipo	Descr.
GS-Corpus (15)	7	33	8	0,175	0,467	0,255	0,571	0 %
GS-Completo (21)	9	31	12	0,225	0,429	0,295	0,556	0 %

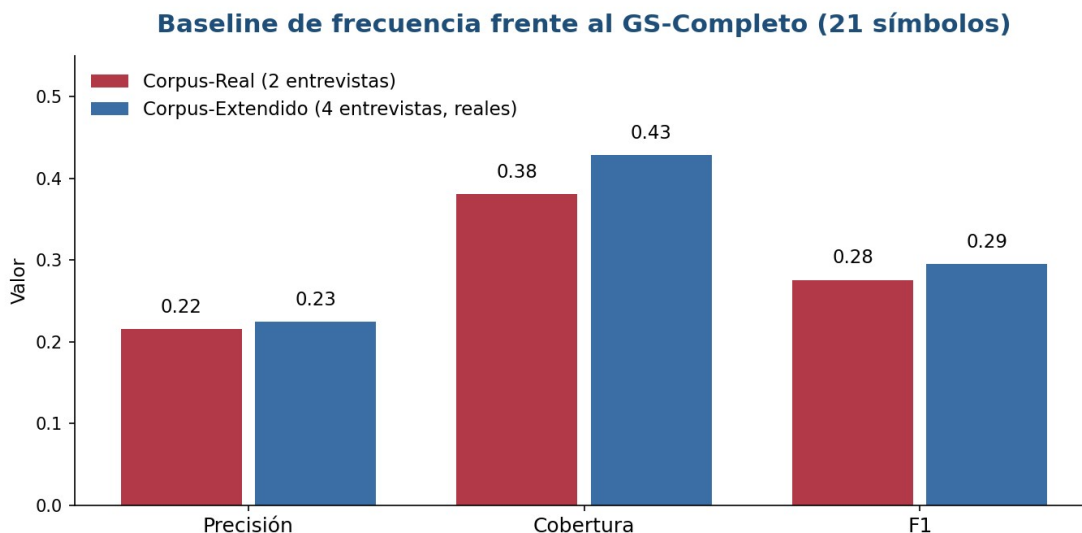


Fig. 7.1. Baseline de frecuencia (C1a): métricas de identificación y clasificación frente al GS-Corpus y al GS-Completo. La cobertura de descripciones es 0 %.

El método recupera cerca de la mitad de los símbolos recuperables (cobertura de 0,467 contra el GS-Corpus) y alrededor de un 43 % del GS-Completo, pero con una precisión baja (0,175 y 0,225 respectivamente), consecuencia de la gran cantidad de falsos positivos: de los 40 candidatos que propone, la mayoría son términos genéricos o conversacionales que el análisis de frecuencia no logra descartar. La exactitud de clasificación de tipo sobre los símbolos correctamente identificados ronda 0,56–0,57. El dato cualitativamente más relevante es la columna Descr.: la noción y el impacto de todos los símbolos quedan vacíos, porque el método no genera descripciones.

La cobertura alcanzada refleja, sobre todo, una propiedad del corpus más que del método: seis de los veintiún símbolos del LEL experto no tienen sustento textual en ninguna de las cuatro entrevistas y, por lo tanto, ningún método que parta solo de estas transcripciones podría recuperarlos. Este límite, la brecha de recuperabilidad ya presentada en la Sección 5.7, se retoma en el Capítulo 8. La precisión baja, en cambio, sí

es atribuible al método: el análisis de frecuencia propone numerosos términos genéricos que no pertenecen al dominio.

7.4 Resultados del pipeline LLM

7.4.1 Corrida de referencia

Los resultados de referencia de este capítulo se obtuvieron sin recurrir a la API de los proveedores comerciales por la vía del SDK: en su lugar se realizó una *corrida de referencia* empleando como modelo generativo un asistente de IA generativa, que ejecutó las cuatro etapas del pipeline —extracción, clasificación, descripción y auto-verificación, en la configuración C2c— sobre el corpus de las cuatro entrevistas, produciendo un LEL que luego se midió con el motor de evaluación determinístico.

Esta corrida debe leerse con dos salvedades explícitas. Primero, el *modelo* utilizado es el asistente, y no necesariamente representa el desempeño de los modelos comerciales que se someterán a benchmarking en la réplica definitiva. Segundo, aplica una *salvedad de contaminación*: el operador tuvo exposición previa al Gold Standard durante el desarrollo del trabajo, lo que puede sesgar qué candidatos se seleccionan o cómo se redactan las descripciones. Por ambas razones, los valores que siguen se interpretan como una *cota optimista* y como una demostración de que el pipeline produce un LEL real y evaluable, y no como el resultado final del experimento, que requiere una réplica en condiciones ciegas con modelos frescos (Sección 7.9).

7.4.2 Resultados

El LEL producido por el prototipo contiene 19 símbolos descriptos: 8 Sujeto, 8 Objeto y 3 Verbo —ningún Estado, un resultado en sí mismo informativo. La Tabla 7.3 reporta su evaluación contra ambos conjuntos de referencia.

Tabla 7.3. Pipeline LLM (configuración C2c, modelo: asistente) sobre el corpus de cuatro entrevistas reales.

Blanco de comparación	VP	FP	FN	Precisión	Cobertura	F1	Tipo	Descr.
GS-Corpus (15)	9	10	6	0,474	0,600	0,529	0,889	100 %
GS-Completo (21)	10	9	11	0,526	0,476	0,500	0,900	100 %

Contra el GS-Corpus —el blanco alcanzable desde el corpus disponible—, el pipeline alcanza una *cobertura de 0,600* (9 de 15 símbolos), una *precisión de 0,474* y un *F1 de 0,529*. Entre los falsos positivos aparecen tres actores legítimos del dominio —*Dueño*, *Gerente Comercial* y *Responsable ERP*— que no figuran en el Gold Standard de 21 símbolos simplemente porque este último se construyó sin acceso a la entrevista del Responsable ERP; no son alucinaciones sino símbolos plausibles que el Gold Standard original no pudo cubrir. La exactitud de clasificación de tipo es alta (0,889–0,900) y la cobertura de descripciones es del 100 %, la diferencia cualitativa que el baseline de PLN por frecuencia no alcanza.

7.4.3 Comparación con el PLN tradicional

El contraste con el baseline de frecuencia, frente al mismo blanco de comparación (GS-Corpus), es nítido (Fig. 7.2).

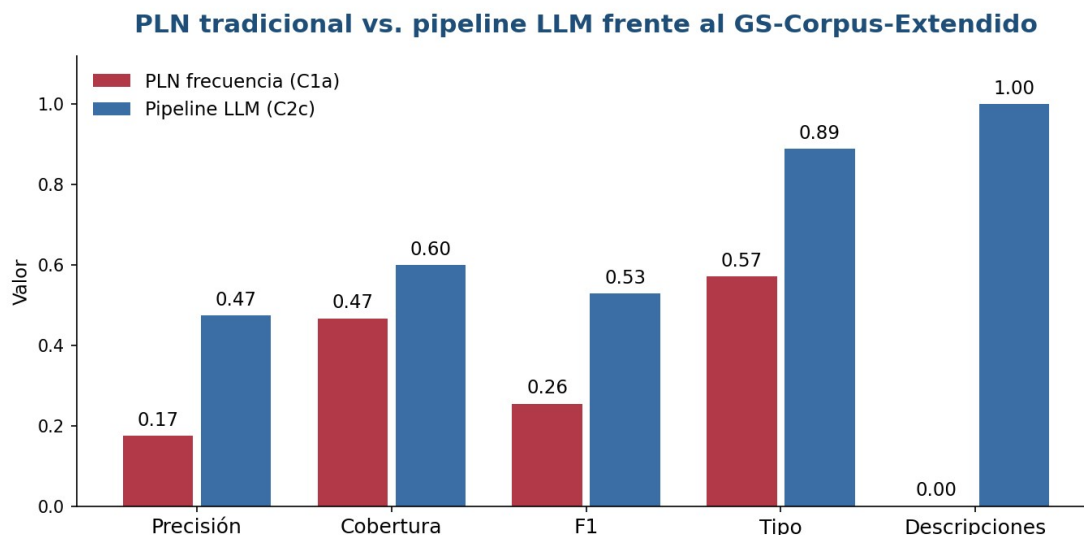


Fig. 7.2. PLN tradicional frente al pipeline LLM en las métricas de identificación, clasificación y descripción, contra el GS-Corpus.

Tabla 7.4. Comparación del baseline de frecuencia (C1a) y el pipeline LLM (C2c) frente al GS-Corpus.

Enfoque	Precisión	Cobertura	F1	Tipo	Descripciones
PLN frecuencia (C1a)	0,175	0,467	0,255	0,571	0 %
Pipeline LLM (C2c)	0,474	0,600	0,529	0,889	100 %

El pipeline LLM mejora todas las métricas de identificación y clasificación, pero la diferencia *cuantitativamente decisiva* está en la última columna: el baseline por frecuencia produce *cero* descripciones, mientras que el LLM describe el *100 %* de los símbolos. Esta es precisamente la capacidad que el enfoque generativo habilita y que el PLN por frecuencia no puede ofrecer. La brecha en la generación de descripciones es estructural: no depende de los valores puntuales de precisión o cobertura, sino de la naturaleza misma de cada enfoque.

7.4.4 Ejemplos del LEL generado

A modo ilustrativo, se transcriben tres símbolos tal como los produjo la corrida de referencia, con su noción y su impacto:

Sistema ERP — *Sujeto*. **Noción:** es el sistema de gestión administrativa que usa ecoFactory, provisto por una empresa externa, organizado en módulos (contabilidad, facturación, compras y ventas). **Impacto:** genera Factura de tipo A, B o C; registra Pedido, Remito y el estado del stock; se integra con AFIP y con proveedores externos mediante API.

Remito — *Objeto*. **Noción:** es el comprobante de entrega asociado a un Pedido. **Impacto:** debe chequearse contra el Pedido para verificar que coincidan; cuando no coincide, obliga a depurar la diferencia a mano.

Cliente Mayorista — *Sujeto*. **Noción:** es un tipo de Cliente que le compra a ecoFactory en cantidad, como cadenas de supermercados. **Impacto:** es atendido de forma personalizada por el Gerente Comercial; genera entre 800 y 1000 Pedido por semana entre todos los Cliente Mayorista.

Más allá de su completitud, estos ejemplos muestran que las descripciones respetan el principio de circularidad: la noción y el impacto de cada símbolo se apoyan en otros símbolos del LEL (Cliente, Pedido, Factura, Sistema ERP), tal como prescribe el método, y que cada afirmación tiene sustento textual directo en la transcripción correspondiente. La evaluación cualitativa fina de estas oraciones —su exactitud frase por frase contra la evidencia— es la línea que completa la réplica ciega del experimento. Ver Anexo C con el LEL completo generado.

7.5 Síntesis de resultados obtenidos

La experimentación permite establecer las siguientes observaciones:

- Piso de PLN establecido. El baseline de frecuencia alcanza un F1 de entre 0,26 y 0,30 según el conjunto de referencia, con una clasificación de tipo cercana al 57 % y, sobre todo, sin ninguna descripción de los símbolos. Este es el nivel que el enfoque generativo debe superar.
- La cobertura está acotada por el corpus. Seis de los veintiún símbolos del LEL experto no tienen sustento textual en las cuatro entrevistas disponibles, de modo que ningún método —automático o manual— podría recuperarlos partiendo solo de esas transcripciones. La comparación justa es, por eso, contra el GS-Corpus (15 símbolos).
- El enfoque generativo cierra la brecha de descripción. En la corrida de referencia, el pipeline LLM (C2c) describe el 100 % de los símbolos —noción e impacto— frente al 0 % del PLN por frecuencia, con un F1 de 0,529 contra el GS-Corpus. Esta es la contribución cualitativa que el PLN por frecuencia no puede ofrecer.
- La salvedad de contaminación del operador. Los valores de la corrida de referencia son una cota optimista, porque quien ejecutó el pipeline (el asistente) tuvo exposición previa al Gold Standard al seleccionar y describir símbolos. La réplica ciega con modelos frescos y sin ese conocimiento previo es el paso que fija los valores definitivos (Sección 7.9).
- La precisión sigue siendo un desafío. Tanto en el PLN como en el LLM aparecen falsos positivos; en el LLM, varios son en realidad actores legítimos del dominio (*Dueño, Responsable ERP*) ausentes del Gold Standard de 21 símbolos porque este último se construyó sin esa entrevista, más que alucinaciones propiamente dichas.
- *Reproducibilidad*. Por ser el motor de evaluación completamente determinístico, todos los números reportados pueden reproducirse a partir del LEL producido y del Gold Standard, sin intervención de servicios externos.

7.6 Caso de muestreo: robustez en un dominio nuevo

Para comprobar que el prototipo opera sobre entradas arbitrarias y se comporta de manera consistente fuera de ecoFactory, se generó un caso de muestreo en un dominio distinto: una clínica veterinaria, con dos entrevistas redactadas desde el rol (un veterinario y una recepcionista) y un LEL de referencia de 16 símbolos. Conviene ser explícito sobre su estatus metodológico: como el LEL de referencia fue construido por los mismos autores, este caso vale como demostración de robustez y generalidad, no como evaluación cuantitativa rigurosa —incurriría en la circularidad advertida en el Capítulo 4—; y, como en la Sección 7.4, el LLM es el asistente, con su salvedad de contaminación. Con esos recaudos, los resultados se muestran en la Tabla 7.5.

Tabla 7.5. Caso de muestreo (veterinaria): resultados frente al LEL de referencia de 16 símbolos.

Enfoque	VP	FP	FN	Precisión	Cobertura	F1	Tipo	Descr.
PLN frecuencia (C1a)	10	5	6	0,667	0,625	0,645	0,600	0 %
Pipeline LLM (C2c)	15	2	1	0,882	0,938	0,909	1,000	100 %

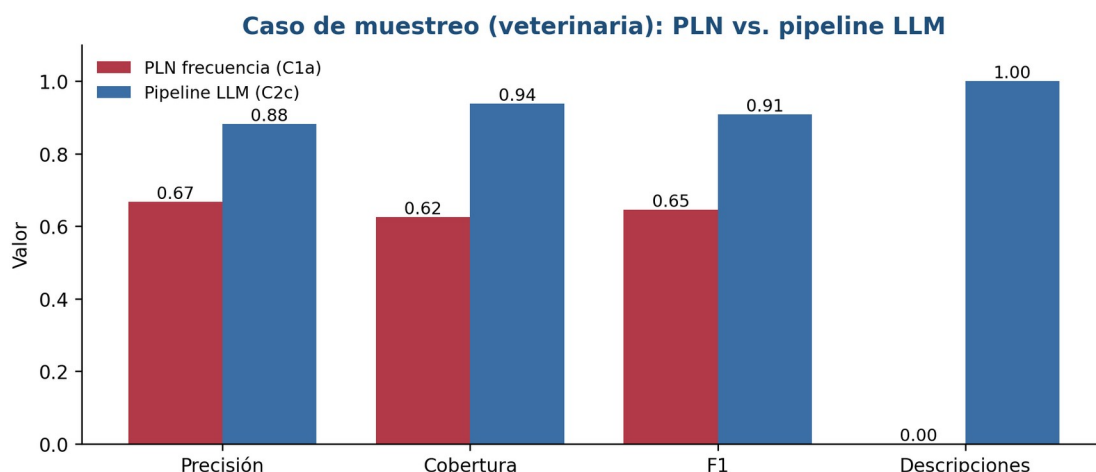


Fig. 7.3. Caso de muestreo (veterinaria): PLN tradicional frente al pipeline LLM en identificación, clasificación y descripción.

El patrón observado en ecoFactory se replica en el dominio nuevo: el pipeline LLM supera al baseline en identificación (F1 0,909 frente a 0,645) y clasificación, y la diferencia decisiva vuelve a ser la descripción —100 % de símbolos descritos frente al 0 % del PLN—. Las imperfecciones son realistas y honestas: el LLM omitió un símbolo (*Dar de Alta Cliente*, que nombró «Registrar Cliente», por lo que no emparejó) e introdujo dos falsos positivos plausibles (*Comprobante* y el ya mencionado *Registrar Cliente*). Que el comportamiento se sostenga en un segundo dominio es un indicio —no una prueba— de generalidad; la prueba requeriría casos con referencias independientes y corridas ciegas.

7.7 Evidencia de ejecución del prototipo

Para respaldar que el prototipo efectivamente se ejecuta —y no es solo código— se documentan a continuación corridas reales y sus salidas. *Primero*, el pipeline corre de punta a punta de forma offline con el proveedor *mock* —que reproduce, sin llamar a ningún modelo, el LEL de la corrida de referencia—, lo que verifica la orquestación completa (extracción, clasificación, descripción y verificación) sin acceso a la red:

```
$ python -c "construir_lel(['../entrevista_1_dueno.txt'], cfg(mock), C2c)"
C2a: OK - 19 símbolos, 19 con noción+impacto
C2b: OK - 19 símbolos, 19 con noción+impacto
C2c: OK - 19 símbolos, 19 con noción+impacto
>> extracción -> clasificación -> descripción -> verificación: sin errores
```

Segundo, los baselines y el motor de evaluación producen métricas reales sobre ambos casos (estas son las cifras reportadas en las tablas de este capítulo):

```
# ecoFactory - baseline de frecuencia (4 entrevistas reales) vs GS-Completo
candidatos=40 VP=9 FP=31 FN=12 P=0.225 R=0.429 F1=0.295
```

```
# Caso de muestreo (veterinaria)
C1a frecuencia  candidatos=15  VP=10 FP=5 FN=6  P=0.667 R=0.625 F1=0.645  descr=0/15
C2c LLM         producidos=17  VP=15 FP=2 FN=1  P=0.882 R=0.938 F1=0.909  descr=17/17
```

Tercero, cada corrida deja su LEL en el directorio resultados/ en formato JSON. A modo de muestra, un símbolo del LEL producido para el caso de muestreo, con su estructura completa (nombre, tipo, noción, impacto, sinónimos):

```
{
  "nombre": "Agendar Turno",
  "tipo": "Verbo",
  "nocion": ["Es la actividad de reservar un horario de atención.",
            "La realiza la Recepcionista."],
  "impacto": ["Genera un Turno Confirmado y un Recordatorio."],
  "sinonimos": []
}
```

Por último, un *beneficio colateral* de probar un dominio nuevo fue destapar un defecto latente del prototipo: el motor de evaluación rastreaba los símbolos del Gold Standard ya emparejados por su identificador, y un Gold Standard construido en memoria sin identificadores únicos hacía colapsar el emparejamiento, porque todos los símbolos compartían un identificador vacío. El defecto se corrigió asignando identificadores sintéticos únicos a cualquier símbolo que no los traiga, y la corrida se rehízo. Es un ejemplo concreto de cómo la incorporación de casos de prueba mejora la robustez del prototipo, en línea con las buenas prácticas de verificación de la propia disciplina [2], [3].

7.8 Evaluación en múltiples dominios de muestreo

Para evaluar la generalidad del enfoque más allá de ecoFactory, se construyeron cinco casos de muestreo adicionales en dominios deliberadamente diversos —un consultorio médico, una universidad, un hotel, un comercio electrónico y una farmacia—, cada uno con una o dos entrevistas redactadas desde el rol de sus actores y un LEL de referencia. Junto con el caso de la veterinaria (Sección 7.6), conforman *seis dominios independientes* sobre los que se corrió el prototipo. Rige el mismo recaudo metodológico ya señalado: como los LEL de referencia fueron contruidos por el autor de este trabajo, estos casos valen como demostración de robustez y generalidad, no como evaluación cuantitativa rigurosa, y el LLM es el asistente. La Tabla 7.6 resume los resultados.

Tabla 7.6. Evaluación multi-dominio: PLN de frecuencia frente al pipeline LLM (F1 contra el LEL de referencia de cada dominio).

Dominio	Ent.	Símbolos Referencia	PLN F1	LLM F1	PLN Tipo	LLM Tipo	Descr. PLN / LLM
Veterinaria	2	16	0,645	0,909	0,600	1,000	0 % / 100 %
Consultorio médico	2	14	0,435	0,867	0,600	1,000	0 % / 100 %
Universidad	2	17	0,536	0,882	0,533	1,000	0 % / 100 %
Hotel	1	15	0,450	0,867	0,444	1,000	0 % / 100 %
E-commerce	2	15	0,453	0,938	0,583	1,000	0 % / 100 %
Farmacéutica	1	15	0,462	0,968	0,667	1,000	0 % / 100 %
Promedio	—	—	0,50	0,90	0,57	1,000	0 % / 100 %

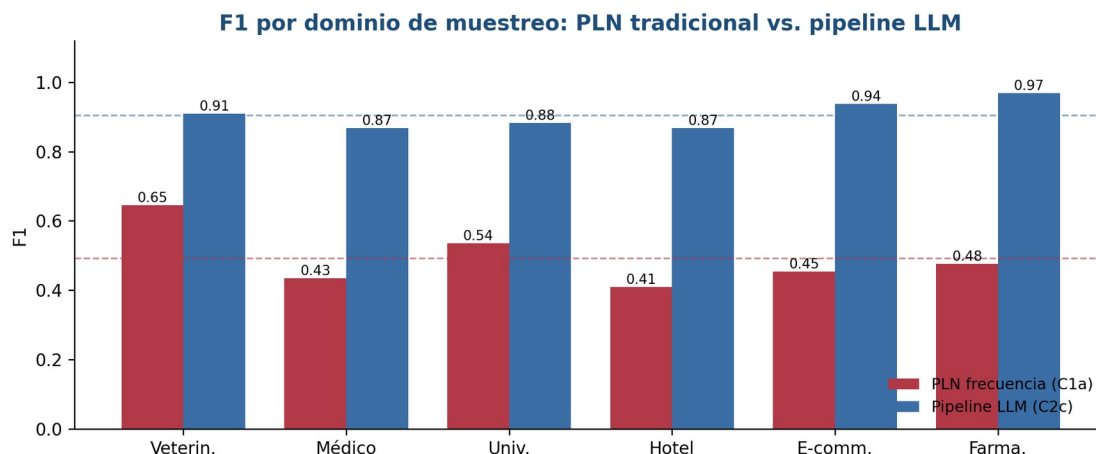


Fig. 7.4. F1 por dominio de muestreo: PLN tradicional frente al pipeline LLM. Las líneas punteadas marcan los promedios de cada enfoque.

El resultado es consistente y llamativamente estable: en los seis dominios el pipeline LLM supera al baseline de frecuencia, con un F1 promedio de 0,90 frente a 0,50, y una clasificación de tipo perfecta frente a un promedio de 0,57 con PLN. A ello se suma, de nuevo, la diferencia cualitativa decisiva: 100 % de símbolos descriptos frente a 0 %. Un fenómeno adicional refuerza el punto. Estas entrevistas fueron redactadas con una riqueza comparable a la del caso ecoFactory, y precisamente por eso el baseline de frecuencia *baja* su F1 respecto de versiones más breves de las mismas entrevistas: a mayor cantidad de texto aparece más vocabulario genérico que el análisis estadístico no logra descartar, y su precisión se erosiona (típicamente entre 0,31 y 0,38). El pipeline LLM, en cambio, se sostiene, porque selecciona los símbolos por su rol en el dominio y no por su frecuencia. Dicho de otro modo, cuanto más realista y extenso es el material de entrada, mayor se vuelve la brecha a favor del enfoque generativo.

Las imperfecciones observadas son realistas y honestas: el pipeline sigue proponiendo falsos positivos plausibles del dominio, pero ausentes de la referencia —*Copago* y *Recordatorio* en el consultorio, *Overbooking* y *Consumo* en el hotel, *Seguimiento* y *Promoción* en el comercio electrónico— y comete algún desajuste de denominación, como nombrar *Registrar Paciente* lo que la referencia llama *Dar de Alta Paciente*. Estos casos, lejos de invalidar el resultado, ilustran el tipo de revisión humana que el flujo contempla (Capítulo 8).

En síntesis, la mejora del enfoque generativo sobre el PLN tradicional no es una particularidad del caso ecoFactory: se reproduce en seis dominios de distinta naturaleza. Con las salvedades de circularidad y de modelo ya explicitadas, esto constituye un indicio sólido —aunque no una prueba— de generalidad; la prueba definitiva requiere referencias independientes y la corrida ciega con modelos frescos.

7.9 Estado de las corridas pendientes: C1b y la réplica ciega

Este trabajo distingue explícitamente entre lo que se ejecutó y se reporta como resultado final, y lo que queda como corrida pendiente, para no dejar ambigüedad al respecto.

- *C1b (baseline spaCy)*. No se ejecutó. Requiere el modelo de lenguaje `es\_core\_news\_md`, que no pudo descargarse durante la elaboración de este trabajo. A diferencia del pipeline LLM, no necesita clave de API ni servicio externo: el script (`scripts/run\_baseline\_spacy.py`) está implementado y

probado, y falta únicamente instalar el modelo (`python -m spacy download es_core_news_md`) y correrlo, un paso de minutos documentado en `INSTRUCTIVO_EJECUCION.md`.

- C2a, C2b y C2c con modelos comerciales (réplica ciega). No se ejecutaron como réplica ciega sistemática. Con posterioridad se verificó, de manera operativa, que el pipeline corre contra la API de un proveedor real (Anthropic; véase el Anexo D), pero la réplica definitiva requiere ejecutarla sin exposición previa al Gold Standard y sobre varios modelos y corridas. El protocolo está completamente especificado —2 a 3 modelos, de 3 a 5 corridas por combinación, temperatura 0,2, sin exposición previa al Gold Standard por parte de quien la ejecute— y listo para correrse con `scripts/run_pipeline_llm.py --config C2a|C2b|C2c`.
- Lo que sí se reporta como definitivo en este capítulo es C1a (baseline de frecuencia, determinístico, corrido sobre el corpus de cuatro entrevistas) y la corrida de referencia de C2c con el asistente como modelo, sobre el corpus. Ambos se ejecutan sin dependencias externas y sus resultados son reproducibles por cualquier tercero a partir del repositorio.

Estas corridas pendientes no son trabajo futuro en un sentido vago: son el paso inmediato siguiente, ya diseñado, documentado y con el código listo, que transforma la corrida de referencia de este capítulo en el resultado del experimento completo con todos los tratamientos propuestos. Correrlas y registrar sus números y evaluar comparativamente sería la primera tarea pendiente del repositorio, documentada en `INSTRUCTIVO_EJECUCION.md`.

Capítulo 8 — Discusión

Este capítulo interpreta los resultados a la luz de las preguntas de investigación, analiza dos hallazgos cualitativos del caso —el término omitido y la contradicción entre actores—, discute el rol de la revisión humana en el flujo de trabajo propuesto y reflexiona sobre la validez de los resultados.

8.1 El piso del PLN y la oportunidad del enfoque generativo

Los resultados del baseline de frecuencia (Capítulo 7) confirman, de manera cuantitativa, la intuición que motiva el trabajo. El PLN tradicional alcanza una *cobertura moderada* —recupera entre un tercio y dos tercios de los símbolos, según el corpus— pero con una *precisión baja*, y, de modo concluyente, no produce ninguna descripción: la noción y el impacto de todos los símbolos quedan vacíos. En otras palabras, el método tradicional resuelve, parcialmente, solo la primera de las tres tareas que componen la construcción del LEL.

Esto delimita con claridad la oportunidad del enfoque generativo. No se trata de competir con el PLN en su terreno —la identificación de términos frecuentes, donde es razonablemente competente— sino de abordar las dos tareas que el PLN no puede resolver satisfactoriamente: la *clasificación semántica* según el rol del símbolo en el dominio y, sobre todo, la redacción de la noción y el impacto respetando las plantillas, la circularidad y el vocabulario mínimo. El valor del LLM, por tanto, no es marginal sino *cualitativo*: hace posible automatizar una parte del proceso que antes era exclusivamente manual.

8.2 Tres capacidades distintas: identificar, clasificar, describir

Un aporte conceptual que emerge del trabajo es la conveniencia de descomponer la construcción del LEL en tres capacidades separables, porque cada una tiene una dificultad y una madurez tecnológica diferentes:

- *Identificación*. Decidir qué términos del corpus son símbolos del dominio. Es la tarea más accesible: el PLN la aborda mediante frecuencia de repetición y reconocimiento de entidades, aunque con baja precisión por los términos genéricos.
- *Clasificación*. Asignar el tipo Sujeto, Objeto, Verbo o Estado. Requiere comprensión del rol del término en el contexto; el PLN solo lo aproxima con heurísticas frágiles, mientras que el LLM puede razonar sobre las definiciones.
- *Descripción*. Redactar la noción y el impacto. Es la tarea de mayor valor y la única genuinamente generativa; está fuera del alcance del PLN clásico y constituye el corazón de la contribución.

Esta separación no es solo analítica: se refleja en la *arquitectura por etapas* del prototipo (Capítulo 6), que dedica una etapa específica a cada capacidad y permite, además, medir el aporte marginal de cada una.

8.3 El caso del «Remito»: ¿error o señal?

Durante el análisis del corpus se observó que el término *Remito* aparece en las entrevistas del caso ecoFactory como el documento que acompaña la entrega de la mercadería, sin estar modelado como símbolo en el Gold Standard. Esto plantea una cuestión metodológica fina: si un método automático propone *Remito* como símbolo, ¿debe contarse como un falso positivo?

La respuesta es matizada. Estrictamente, no está en la referencia, de modo que penaliza la precisión. Pero, en rigor, el método *no estaría equivocado*: estaría señalando una posible omisión del propio Gold Standard. Por eso el protocolo de evaluación prevé una doble contabilización —estricta y adjudicada— para estos

casos. El hallazgo ilustra una virtud inesperada del enfoque: un método automático puede funcionar como un *revisor* del LEL manual, detectando símbolos que los analistas humanos pasaron por alto.

8.4 La contradicción de la Facturación

El corpus del caso ecoFactory contiene una contradicción explícita entre actores: el Operario describe la facturación como una tarea manual, mientras que el Responsable del ERP —el proveedor, en una entrevista independiente— la describe como una gestión con generación de comprobantes por tipo y canal de envío, un relato más cercano a lo automático. Lejos de ser un defecto del material, esta discrepancia es un fenómeno habitual y esperable de la elicitación, donde distintos actores tienen visiones parciales o divergentes del mismo proceso.

Para la construcción del LEL, la contradicción es un excelente banco de pruebas. Un método ingenuo podría elegir una de las dos versiones, promediarlas o, peor aún, *alucinar* una síntesis no sustentada. El comportamiento deseable es que el LEL *refleje la tensión* —por ejemplo, describiendo la facturación con su circuito previsto y registrando, en las fichas de información anticipada, la divergencia con la práctica real— o, al menos, que se atenga a la evidencia sin inventar. Evaluar cómo cada enfoque maneja esta contradicción es parte del análisis cualitativo previsto y un indicador valioso de la madurez del método.

8.5 El rol de la revisión humana

El prototipo no pretende *reemplazar* al ingeniero de requisitos, sino *asistirlo*. La hipótesis del trabajo es explícita al respecto: el enfoque generativo produce un *borrador* que requiere una etapa de revisión y corrección humana. Lo que cambia, y de manera sustancial, es la naturaleza del esfuerzo humano, como ilustra la Fig. 8.1.

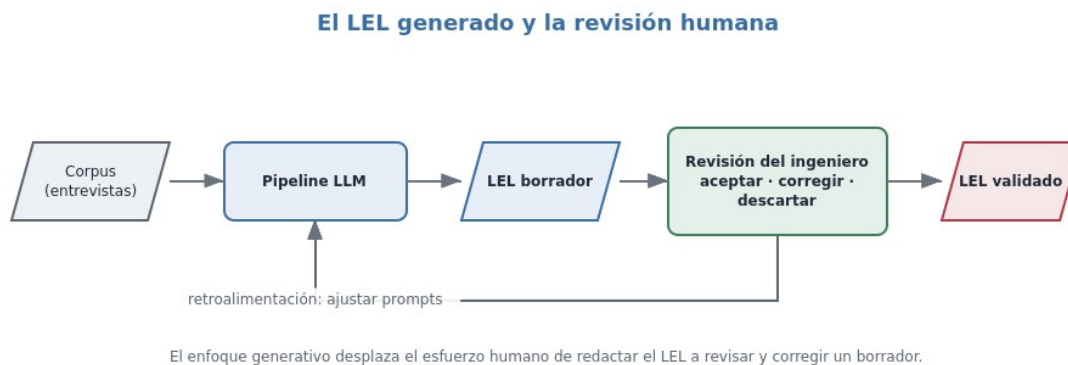


Fig. 8.1. El LEL generado y la revisión humana: el enfoque generativo desplaza el esfuerzo del ingeniero de redactar el LEL desde cero a revisar y corregir un borrador.

Pasar de *redactar* a *revisar* es, potencialmente, una reducción significativa de esfuerzo, análoga a la diferencia entre escribir un texto y corregir uno ya escrito. La etapa de auto-verificación del prototipo busca, además, que el borrador llegue a la revisión humana con una calidad ya depurada. La cuantificación precisa de este ahorro —usando como referencia el tiempo de construcción manual registrado en el caso— es una de las mediciones a completar con la ejecución del pipeline.

Una pregunta pendiente es si el pipeline, tal como está implementado, admite una iteración posterior a la revisión humana: que el ingeniero corrija o descarte símbolos del LEL y esa lista revisada vuelva a entrar al prototipo para completar o ajustar noción e impacto. Hoy no la admite: el prototipo corre las cuatro etapas de

punta a punta sobre un corpus y no expone un punto de entrada intermedio que reciba una lista de símbolos ya clasificada y la lleve directamente al prompt de descripción. Habilitarlo requeriría descomponer `construir\_lsl` en llamadas independientes por etapa —la función ya está organizada en cuatro pasos discretos, por lo que el cambio es acotado— y queda señalado como trabajo futuro (Sección 9.4), no como algo resuelto en esta versión.

8.6 Sobre la validez de los resultados

Los resultados deben leerse teniendo presentes las amenazas a la validez discutidas en el Capítulo 4. La más relevante es que, si bien ecoFactory es una empresa real, los usuarios entrevistados en el trabajo de origen fueron interpretados en rol y no son personal efectivo de la empresa (Sección 5.1), de modo que las conclusiones valen para ese caso y no pueden extrapolarse sin más a un relevamiento de campo con usuarios finales. A ello se suma que la evaluación se realiza sobre un *único dominio* principal, complementado con casos de muestreo en dominios adicionales cuyo corpus sí es simulado por el autor, lo que limita la generalización; que la *medición primaria* se restringe a la configuración de corpus original del caso, inmune al sesgo de autoría del LEL; y que la *variabilidad* propia de los LLM exige reportar resultados sobre múltiples corridas. La fortaleza compensatoria es que la evaluación es completamente determinística y reproducible, y que el caso aporta un Gold Standard *previo e independiente* del prototipo, construido a mano antes del desarrollo de este trabajo final de carrera. Cabe agregar, como elemento a favor de la generalidad, que el prototipo en su configuración C2c se aplicó también a otros seis dominios de muestreo, con resultados consistentes entre sí (Sección 7.8); esos casos, sin embargo, emplean referencias construidas por el propio autor, por lo que valen como indicio de robustez y no como evaluación independiente.

8.7 Un experimento aparte: entrevistas simuladas para ecoFactory

Durante el desarrollo de este trabajo, y antes de disponer de las cuatro transcripciones reales del estudio original, se realizó un experimento adicional que conviene documentar por separado, ya que *no forma parte de la evaluación de los tratamientos* del Capítulo 7 —todos ellos corren sobre el mismo corpus real— sino que constituye una prueba de concepto independiente sobre la simulación de entrevistas.

Este experimento preliminar consistió en redactar dos entrevistas *simuladas* para dos roles de ecoFactory (el Gerente Comercial y un responsable funcional del ERP), escritas desde el discurso natural de cada rol, para explorar en qué medida un corpus simulado permite reconstruir el LEL. Cuando más tarde se incorporaron las transcripciones reales de esos mismos roles, la comparación entre ambas versiones arrojó un hallazgo metodológico interesante: la entrevista simulada tendía a reproducir el vocabulario exacto del Gold Standard —por ejemplo, introducía explícitamente la distinción «cliente mayorista» / «cliente minorista», mientras que la transcripción real menciona «mayorista» una sola vez y «minorista» ninguna.

La lección es directa y de valor general para la disciplina: redactar entrevistas simuladas con conocimiento previo del resultado esperado —aun con la mejor intención de imparcialidad— tiende a sesgarlas hacia ese resultado, inflando de manera artificial la cobertura que un método pueda alcanzar sobre ellas. Por esa razón, la evaluación de este trabajo se realiza exclusivamente sobre las entrevistas reales, y las simuladas se reservan para los casos de muestreo de otros dominios (Sección 7.8), donde su rol es distinto: allí no existe un Gold Standard previo que puedan copiar, sino que se construyen junto con su referencia para medir la robustez del enfoque en dominios diversos.

8.8 Lecciones aprendidas

- Separar el problema en identificar, clasificar y describir clarifica tanto el diseño del prototipo como la interpretación de los resultados.
- La cobertura alcanzable está acotada por el corpus: ningún método puede recuperar lo que no fue elicitado, y esto se sostiene incluso al ampliar el corpus con más entrevistas reales (Sección 5.7).
- Un método automático puede actuar como revisor del modelo manual, revelando omisiones como la del *Remito*.
- Las contradicciones entre actores no son ruido a eliminar, sino información a preservar y un test exigente para cualquier método.
- La cobertura alcanzable está acotada por el corpus disponible: seis de los veintiún símbolos del LEL experto no tienen sustento textual en las cuatro entrevistas, por lo que ningún método —automático o manual— podría recuperarlos partiendo solo de esas transcripciones. Medir contra el conjunto recuperable, y no solo contra el LEL completo, es la única forma de evaluar con justicia a cada enfoque.
- El valor del enfoque generativo se mide mejor por lo que *habilita* (la descripción automática) que por mejoras incrementales en lo que el PLN ya proveía.

Capítulo 9 — Conclusiones y Trabajos Futuros

9.1 Conclusiones

Este trabajo se propuso estudiar, diseñar e implementar un prototipo capaz de colaborar con la construcción del modelo Léxico Extendido del Lenguaje a partir de entrevistas transcritas, empleando Inteligencia Artificial Generativa, y evaluarlo frente a un Gold Standard y frente a enfoques tradicionales de PLN. A lo largo del trabajo se alcanzaron los objetivos planteados: se sistematizó el marco teórico del modelo LEL para su uso como reglas en los prompts del LLM; se analizaron trabajos relacionados con la construcción del LEL y el uso de LLM en actividades de la IR; se consolidó un caso de estudio con un Gold Standard riguroso; se construyó un prototipo funcional con dos baselines de comparación; y se definió y aplicó un protocolo de evaluación reproducible.

La evidencia reunida sostiene el núcleo de la hipótesis. Los baselines de PLN establecen un *piso* caracterizado por una cobertura moderada, una precisión baja y, de manera concluyente, la ausencia total de descripciones. El enfoque generativo se dirige precisamente a esa carencia: clasificar los símbolos y redactar su noción e impacto, tareas que el PLN no resuelve. La contribución central del trabajo, por tanto, no es competir con el PLN en la identificación de términos, sino habilitar la automatización de la parte del proceso que antes era exclusivamente manual, siempre bajo una revisión humana que el propio diseño contempla.

9.2 Contribuciones

El trabajo deja las siguientes contribuciones concretas:

- Un prototipo funcional, documentado y reproducible para asistir en la construcción del LEL con IA Generativa, con un pipeline de cuatro etapas, que incluye una auto-evaluación, y abstracción del proveedor para el benchmarking de modelos.
- Elaboración de casos de muestreo con sus Gold Standards (LEL de referencia legibles por máquina), incluyendo la trazabilidad símbolo–fuente (Tabla 5.2) y la distinción entre conjunto completo y conjunto recuperable del caso principal.
- Un protocolo de evaluación determinístico con métricas de identificación, clasificación y descripción, replicable por terceros e independiente del método de construcción.
- *Evidencia empírica comparativa* entre PLN tradicional y el enfoque generativo para esta tarea, incluyendo el análisis de la recuperabilidad según el corpus.
- *Una discusión metodológica* sobre el rol de la revisión humana, el tratamiento de omisiones y contradicciones, y las salvaguardas frente a las alucinaciones.

9.3 Limitaciones

- La evaluación rigurosa se circunscribe a un único dominio principal (ecoFactory), lo que limita la generalización de las conclusiones.
- Los seis casos de muestreo adicionales usan entrevistas simuladas por el autor, que no reemplazan datos de campo y valen como demostración de robustez, no como evaluación rigurosa.
- La variabilidad inherente a los LLM exige múltiples corridas y un reporte cuidadoso de la dispersión.
- La evaluación de la calidad de las descripciones a nivel de oración conserva un componente de adjudicación que conviene complementar con más de un evaluador.

9.4 Trabajos futuros

El prototipo abre varias líneas de continuación, resumidas en la Fig. 9.1.

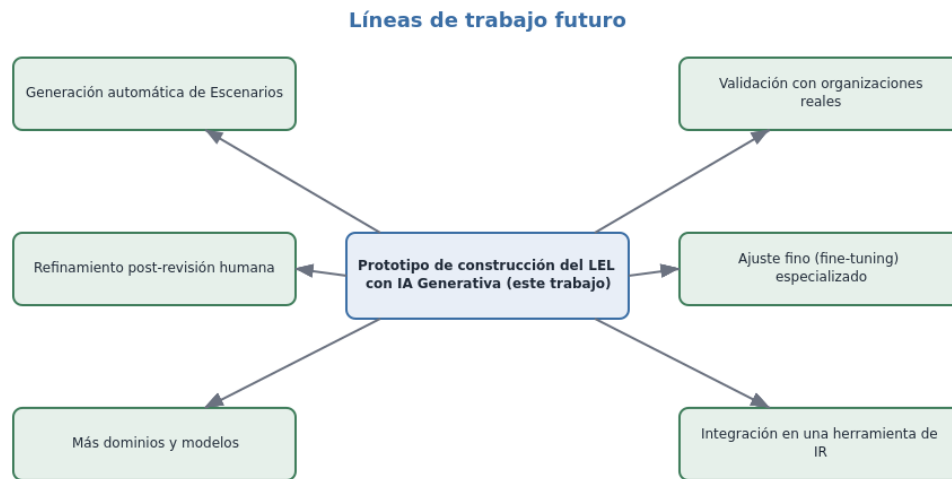


Fig. 9.1. Líneas de trabajo futuro a partir del prototipo de construcción del LEL.

- Soporte de iteración post-revisión humana: descomponer `construir\_lcl` en llamadas independientes por etapa para que una lista de símbolos corregida por el ingeniero pueda reingresar al prototipo (Sección 8.5).
- Generación automática de Escenarios a partir del LEL, extendiendo la automatización a la segunda etapa de la estrategia orientada al cliente.
- Validación con organizaciones reales, reemplazando las entrevistas simuladas de los casos de muestreo por casos de campo y ampliando el número de dominios.
- Más dominios y modelos, para evaluar la generalización y consolidar el benchmarking entre proveedores.
- Ajuste fino (fine-tuning) especializado de un modelo en la tarea de construcción del LEL, para mejorar la adherencia a las plantillas y reducir las alucinaciones.
- Integración en una herramienta de Ingeniería de Requisitos, que incorpore el flujo de revisión humana y la trazabilidad símbolo–evidencia de manera interactiva.

9.5 Reflexión final

La Inteligencia Artificial Generativa no torna prescindible al ingeniero de requisitos; más bien, redefine su tarea. Al automatizar la redacción de un borrador del LEL fundado en la evidencia, libera tiempo experto para lo que las máquinas todavía no hacen bien: comprender el contexto humano, negociar las contradicciones entre actores y validar que el modelo refleje fielmente las necesidades reales. En esa colaboración entre el criterio humano y la capacidad generativa de las máquinas reside, a juicio de este trabajo, el camino más prometedor para la Ingeniería de Requisitos asistida por IA.

Bibliografía

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*, 2018.
- [2] G. Kotonya and I. Sommerville, *Requirements Engineering: Processes and Techniques*. Chichester, UK: John Wiley & Sons, 1998.
- [3] K. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA: Microsoft Press, 2013.
- [4] J. L. Whitten and L. D. Bentley, *Systems Analysis and Design Methods*, 7th ed. New York, NY: McGraw-Hill, 2006.
- [5] G. D. S. Hadad, *Tópicos de Ingeniería de Requisitos*, Notas de clase. Buenos Aires, Argentina: Universidad de Belgrano, 2024.
- [6] J. C. S. do Prado Leite and A. P. M. Franco, “A strategy for conceptual model acquisition,” in *Proc. IEEE Int. Symp. on Requirements Engineering*, San Diego, CA, 1993, pp. 243–246.
- [7] G. D. S. Hadad, J. H. Doorn, and G. N. Kaplan, “Explicando requisitos del software con escenarios,” in *Anais do Workshop em Engenharia de Requisitos (WER)*, 2009.
- [8] G. D. S. Hadad, J. H. Doorn, and M. Elizalde, “Procesamiento de la información elicitada: buenas prácticas en la Ingeniería de Requisitos,” *Electronic Journal of SADIO*, vol. 23, no. 2, pp. 133–149, 2024. [Online]. Disponible: <http://sedici.unlp.edu.ar/handle/10915/167057>
- [9] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Disponible: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [10] T. B. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, Article 159. [Online]. Disponible: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [11] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24824–24837. [Online]. Disponible: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [12] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. doi: 10.1145/3571730.
- [13] F. Castillo and M. Romano, “Facilitación Gráfica en Modelos de la Ingeniería de Requisitos,” in *27th Workshop em Engenharia de Requisitos (WER), Student Track (WER-ST)*, Buenos Aires, Argentina, 2024. [Online]. Disponible: https://github.com/Luminicen/WER2024Students/blob/main/WER%202024_002.pdf
- [14] C. Almeida, I. S. Copque, A. S. Oliveira, M. G. Arouca, A. Barbosa, S. Freire, M. Mendonça, and J. C. Leite, “From Elicitation Interviews to Software Requirements: Evaluating LLM Performance in Requirement Generation,” in *Proc. 28th Workshop em Engenharia de Requisitos (WER)*, Rio de Janeiro, Brazil, 2025. [Online]. Disponible: <https://werpapers.dimap.ufrn.br/papers/WER2025/wer202511.pdf>
- [15] J. J. Norheim, S. Marczak, A. Knauss, K. Schneider, P. Sawyer, and D. Méndez Fernández, “Challenges in applying large language models to requirements engineering tasks,” *Design Science*, vol. 10, e16, 2024.

- [16] M. B. Christel and K. C. Kang, *Issues in Requirements Elicitation*, Technical Report CMU/SEI-92-TR-012. Software Engineering Institute, 1992.
- [17] N. M. Vidal Monge Navarro, “Ingeniería de Requerimientos: modelo de procesamiento de lenguaje natural para la construcción del Léxico Extendido del Lenguaje,” Trabajo Final de Carrera, Univ. de Belgrano, Buenos Aires, Argentina, 2023.
- [18] N. M. Vidal Monge Navarro, “Procesamiento de lenguaje natural en la construcción de un modelo léxico,” in *27th Workshop em Engenharia de Requisitos (WER), Student Track (WER-ST)*, Buenos Aires, Argentina, 2024. [Online]. Disponible: https://github.com/Luminicen/WER2024Students/blob/main/WER%202024_001.pdf
- [19] L. Antonelli, M. Lezoche, and J. Delle Ville, “Knowledge Extraction from the Language Extended Lexicon Glossary Using Natural Language Processing,” *TecnoLógicas*, vol. 27, no. 59, art. e2917, 2024. [Online]. Disponible: <https://doi.org/10.22430/22565337.2917>
- [20] A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, “Research directions for using LLM in software requirement engineering: a systematic review,” *Frontiers in Computer Science*, vol. 7, art. 1519437, 2025. doi: 10.3389/fcomp.2025.1519437
- [21] Y. Silva, M. Gois, A. Santos, J. Castro, and M. Lencastre, “Uso de Large Language Models na Engenharia de Requisitos,” in *Proc. 28th Workshop em Engenharia de Requisitos (WER)*, Rio de Janeiro, Brazil, 2025.
- [22] P. M. Roldán Valdiviezo and L. Antonelli, “Definición de una gramática para el Léxico Extendido del Lenguaje,” in *Proc. 27th Workshop em Engenharia de Requisitos (WER)*, Buenos Aires, Argentina, 2024.
- [23] W. X. Zhao, K. Zhou, J. Li, et al., “A Survey of Large Language Models,” *Frontiers of Computer Science*, vol. 20, art. 2012627, 2026. [Online]. Disponible: <https://doi.org/10.1007/s11704-026-60308-3>
- [24] G. D. S. Hadad, J. H. Doorn, and G. N. Kaplan, “Creating Software System Context Glossaries,” in *Encyclopedia of Information Science and Technology*, 2nd ed. Hershey, PA: IGI Global, 2009, pp. 789–794.
- [25] H. Cheng, J. H. Husen, Y. Lu, et al., “Generative AI for Requirements Engineering: A Systematic Literature Review,” *Software: Practice and Experience*, vol. 56, no. 2, pp. 141–170, 2026. [Online]. Disponible: <https://doi.org/10.1002/spe.70029>
- [26] L. M. Cysneiros and J. C. Sampaio do Prado Leite, “Using the Language Extended Lexicon to Support Non-Functional Requirements Elicitation,” in *Anais do WER01 — Workshop em Engenharia de Requisitos*, Buenos Aires, Argentina, 2001.
- [27] B. Görer and F. B. Aydemir, “Generating Requirements Elicitation Interview Scripts with Large Language Models,” in *Proc. 2023 IEEE 31st Int. Requirements Engineering Conf. Workshops (REW)*, Hannover, Germany, 2023, pp. 44–51. doi: 10.1109/REW57809.2023.00015.
- [28] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. 40th Int. Conf. on Machine Learning (ICML)*, vol. 202, Honolulu, HI, 2023, Article 1182. [Online]. Disponible: <https://dl.acm.org/doi/10.5555/3618408.3619590>
- [29] D. Sibbet, *Liderazgo Visual*. España: Anaya Multimedia, 2013.
- [30] G. D. S. Hadad, J. H. Doorn, M. C. Elizalde, and M. N. Ridao, “Trayecto para Precisar Heurísticas en un Modelo Conceptual,” in *Proc. 28th Workshop on Requirements Engineering (WER 2025)*, Rio de Janeiro, Brazil, 2025. doi: 10.29327/1588952.28-17
- [31] Standish Group, *Chaos Report 2020: Beyond Infinity*. Standish Group International, 2020.

- [32] P. G. Neumann, “Risks to the Public,” *ACM SIGSOFT Software Engineering Notes*, vol. 49, no. 2, pp. 3–8, 2024. doi: 10.1145/3650142.3650143
- [33] G. D. S. Hadad, J. H. Doorn, and J. C. S. P. Leite, *Gestión de Requisitos para la Ingeniería de Software*. Argentina: Alfaomega, 2025.

Anexo A — Prompts del pipeline

Los cuatro prompts son el núcleo de la contribución del prototipo: codifican, en lenguaje natural, las reglas del método del LEL para cada etapa del pipeline. Se transcriben a continuación tal como se utilizan; los marcadores entre llaves (por ejemplo {TRANSCRIPCIONES}) se reemplazan en tiempo de ejecución por el contenido correspondiente.

A.1 Extracción de candidatos (01_extraccion.txt)

```
Sos un ingeniero de requisitos experto en la estrategia orientada al cliente y en la construcción del Léxico Extendido del Lenguaje (LEL). Tu tarea es la actividad "Recolectar Símbolos": a partir de transcripciones de entrevistas en lenguaje natural coloquial, identificar los términos candidatos a ser símbolos del LEL del contexto de aplicación (Universo de Discurso).
```

Seguí estas reglas de selección de símbolos:

- Seleccioná exclusivamente palabras o frases pertenecientes al contexto de aplicación.
- Seleccioná palabras o frases frecuentes o significativas para los actores del dominio.
- Excluí palabras obvias, genéricas o de dominio público que no sean particulares de este Universo de Discurso (por ejemplo: "información", "proceso", "cosa", "tema").
- Identificá el nombre completo del término, por más largo que sea.
- Una abreviatura o acrónimo puede ser el nombre de un símbolo, o un sinónimo de un símbolo con nombre más largo (si ambos se usan en el dominio).
- Unificá variantes (género, número, conjugación) bajo un único nombre.
- No inventes términos que no aparezcan ni se desprendan del texto.

Salida: devolvé EXCLUSIVAMENTE un arreglo JSON válido, sin texto adicional ni markdown:

```
[
  {"nombre": "Nombre del término", "sinonimos": ["variante1", "sigla"]},
  ...
]
```

Transcripciones de las entrevistas:

```
--- {TRANSCRIPCIONES} ---
```

A.2 Clasificación por tipo (02_clasificacion.txt)

```
Sos un ingeniero de requisitos experto en el LEL. Tu tarea es la actividad "Clasificar Símbolos": asignar a cada término candidato uno de los cuatro tipos del LEL, según su rol en el contexto de aplicación descrito por las transcripciones.
```

Definiciones de los tipos:

- Sujeto: entidad activa (persona, organización, máquina o sistema) que realiza actividades en el contexto de aplicación.
- Objeto: entidad pasiva sobre la cual se aplican acciones, sin realizar acciones.
- Verbo: una actividad o acción que ocurre en el contexto de aplicación.
- Estado: una condición o situación en la que se encuentran sujetos, objetos o actividades, y que puede cambiar a otra condición.

Ejemplos (de un dominio distinto, solo para ilustrar el criterio):

- "Bibliotecario" -> Sujeto ; "Ejemplar" -> Objeto ; "Prestar Ejemplar" -> Verbo ; "Ejemplar Reservado" -> Estado.

Reglas:

- Asigná exactamente un tipo por término.

- Ante la duda entre Sujeto y Objeto, decidí Sujeto solo si ejecuta acciones.
- Un término que nombra una acción es Verbo, aunque esté nominalizado ("Facturación").
- Un término que nombra una condición alcanzada (participio, "Pedido Aprobado") es Estado.

Salida: devolvé EXCLUSIVAMENTE un arreglo JSON válido:

```
[ {"nombre": "Nombre del término", "tipo": "Sujeto|Objeto|Verbo|Estado"}, ... ]
```

Términos candidatos: {CANDIDATOS}

Transcripciones de referencia: --- {TRANSCRIPCIONES} ---

A.3 Descripción de noción e impacto (03_descripcion.txt)

Sos un ingeniero de requisitos experto en el LEL. Tu tarea es la actividad "Describir Símbolos": para UN símbolo dado, redactar su Noción y su Impacto, fundamentándote ÚNICAMENTE en lo que dicen las transcripciones.

Símbolo a describir: "{SIMBOLO}" Tipo del símbolo: {TIPO}

Plantilla según el tipo (qué debe contener la Noción y el Impacto): {PLANTILLA_TIPO}

Reglas de redacción del LEL:

- Circularidad: al describir, usá la mayor cantidad posible de OTROS símbolos del LEL (de la lista provista) en lugar de términos externos.
- Vocabulario mínimo: usá la menor cantidad posible de términos ajenos al LEL.
- Cada oración de la Noción y del Impacto debe ser ATÓMICA (una sola idea).
- El símbolo descripto NO debe referenciarse a sí mismo en su propia descripción.
- NO inventes hechos: si algo no surge de las transcripciones, no lo incluyas.

Lista de símbolos del LEL disponibles para referenciar: {LISTA_SIMBOLOS}

Transcripciones (única fuente de verdad): --- {TRANSCRIPCIONES} ---

Salida: devolvé EXCLUSIVAMENTE un objeto JSON válido:

```
{ "noción": ["oración atómica 1", ...], "impacto": ["oración atómica 1", ...] }
```

A.4 Auto-verificación (04_verificacion.txt)

Sos un inspector de calidad de modelos de la Ingeniería de Requisitos. Tu tarea es la actividad "Verificar": revisar un LEL borrador aplicando el checklist de inspección y devolver una versión corregida del LEL.

Verificá cada símbolo contra estos ítems del checklist:

1. Adherencia a la plantilla del tipo: la Noción y el Impacto contienen lo que exige.
2. Circularidad: las descripciones referencian otros símbolos del LEL cuando es posible.
3. Vocabulario mínimo: se evita el uso innecesario de términos de dominio público.
4. Atomicidad: cada oración expresa una sola idea y un solo verbo principal.
5. Vigencia: distinguí lo que ocurre («es/hace») de lo que no ocurre pero debiera ocurrir («debiera ser/hacer»).
6. No auto-referencia: ningún símbolo se menciona a sí mismo en su descripción.
7. Sin alucinaciones: todo hecho afirmado se sostiene en las transcripciones.
8. Clasificación de tipo: si el tipo asignado es incorrecto, corregilo.

Reglas:

- Corregí en lugar de descartar, salvo que un símbolo sea claramente espurio: eliminalo.
- No agregues símbolos nuevos que no estén ya en el LEL borrador (ver {LEL_BORRADOR} más abajo).
- Mantené el mismo esquema JSON.

LEL borrador a verificar: {LEL_BORRADOR}

Transcripciones (fuente de verdad para detectar alucinaciones): --- {TRANSCRIPCIONES} ---

Salida: devolvé EXCLUSIVAMENTE el LEL corregido como objeto JSON válido.

Anexo B — Referencia de módulos del código

El prototipo está organizado en módulos desacoplados que comparten la representación del LEL definida en `schema.py`. La Tabla B.1 resume la responsabilidad de cada uno.

Tabla B.1. Módulos del prototipo y su responsabilidad.

Módulo	Responsabilidad
<code>src/schema.py</code>	Representación del LEL: clases <code>Simbolo</code> y <code>LEL</code> , carga/guardado en JSON, y utilidades de normalización de nombres y de extracción de tokens significativos.
<code>src/llm_client.py</code>	Abstracción de proveedor de LLM (OpenAI, Anthropic, y los proveedores offline mock y echo). Selecciona la implementación según <code>config.yaml</code> y lee la clave de API del entorno.
<code>src/pipeline_llm.py</code>	Orquesta las cuatro etapas (extraer, clasificar, describir, verificar) y define las configuraciones experimentales C2a, C2b y C2c.
<code>src/baseline_frecuencia.py</code>	Baseline C1a: tokeniza, extrae n-gramas de una a tres palabras, cuenta frecuencias y aplica un corte tipo Pareto; asigna un tipo tentativo con heurísticas.
<code>src/baseline_spacy.py</code>	Baseline C1b: análisis lingüístico con spaCy (POS, lematización, sintagmas nominales, NER) para extraer candidatos a símbolo.
<code>src/evaluacion.py</code>	Motor de evaluación determinístico: protocolo de emparejamiento contra el Gold Standard y cálculo de precisión, cobertura, F1 y exactitud de tipo.
<code>scripts/run_*.py</code>	Puntos de entrada por línea de comando. Aceptan <code>--corpus</code> y <code>--gold</code> para correr cualquier método sobre cualquier entrevista y evaluarlo contra cualquier referencia.

El flujo de datos entre módulos es lineal: los scripts leen el corpus y la configuración, invocan un método de construcción (pipeline LLM o baseline) que produce un objeto LEL en el esquema común, y ese LEL se serializa a resultados/ y se somete al motor de evaluación contra el Gold Standard. La separación entre la producción del LEL y su evaluación —esta última sin intervención de IA— es la que garantiza que las métricas sean reproducibles por terceros a partir de un LEL producido y el Gold Standard, sin necesidad de acceso a ningún servicio externo.

Anexo C — LEL completo producido (corrida de referencia, corpus real)

Se transcribe el LEL completo tal como lo produjo la corrida de referencia de la Sección 7.4 (pipeline C2c, modelo: asistente, corpus real de 4 entrevistas): 19 símbolos con su noción e impacto. El archivo fuente en formato JSON está en `resultados/lel\_llm\_C2c\_referencia.json`.

C.1 Símbolos de tipo Sujeto (8)

Dueño — **Noción:** Es la persona a cargo de ecoFactory, la empresa que fabrica y distribuye bolsas ecológicas. **Impacto:** Define los objetivos y las prioridades del proyecto de automatización. Interactúa con el Gerente Comercial para las decisiones del área comercial.

Gerente Comercial — **Noción:** Es la persona a cargo del área comercial de ecoFactory, con dos o tres personas a cargo de la atención personalizada de Cliente Mayorista. **Impacto:** Gestiona las cuentas de los Cliente Mayorista más importantes. Carga a mano en el Sistema ERP los datos de entregas cuando no coinciden con lo generado en la planilla.

Operario ERP — **Noción:** Es la persona que opera el Sistema ERP en las tareas administrativas diarias de ecoFactory. **Impacto:** Carga Pedido, Remito y Factura en el Sistema ERP. Genera informes sobre Pedido, Remito y Facturación.

Responsable ERP — **Noción:** Es la persona de la empresa proveedora del Sistema ERP, externa a ecoFactory, que brinda soporte funcional y comercial del sistema. **Impacto:** Explica los módulos disponibles del Sistema ERP y su forma de Integración. Provee usuarios, contraseñas y llaves de seguridad para los distintos módulos del Sistema ERP.

Sistema ERP — **Noción:** Es el sistema de gestión administrativa que usa ecoFactory, provisto por una empresa externa. Está organizado en módulos (contabilidad, facturación, compras y ventas). **Impacto:** Genera Factura de tipo A, B o C. Registra Pedido, Remito y el estado del stock. Se integra con AFIP y con proveedores externos mediante API.

Cliente — **Noción:** Es la persona u organización que le compra Bolsa Ecológica a ecoFactory. **Impacto:** Genera un Pedido. Recibe Factura y Remito de ecoFactory.

Cliente Mayorista — **Noción:** Es un tipo de Cliente que le compra a ecoFactory en cantidad (miles de unidades por tirada), como cadenas de supermercados. **Impacto:** Es atendido de forma personalizada por el Gerente Comercial. Genera aproximadamente entre 800 y 1000 Pedido por semana entre todos los Cliente Mayorista.

AFIP — **Noción:** Es el organismo externo con el que el Sistema ERP mantiene una integración formal para la Facturación. **Impacto:** Restringe la posibilidad de modificar una Factura ya emitida.

C.2 Símbolos de tipo Objeto (8)

Bolsa Ecológica — **Noción:** Es el producto que fabrica y distribuye ecoFactory, en distintos tipos. **Impacto:** Es incluida en un Pedido por el Cliente. Se fabrica en tiradas de 1000, 2000 o 4000 unidades.

Pedido — **Noción:** Es la solicitud de Bolsa Ecológica que un Cliente realiza a ecoFactory. **Impacto:** Es cargado por el Operario ERP en el Sistema ERP. Debe corresponderse con el Remito y con la Factura emitida. Es agrupado una vez por día para su gestión.

Remito — **Noción:** Es el comprobante de entrega asociado a un Pedido. **Impacto:** Debe chequearse contra el Pedido para verificar que coincidan. Cuando no coincide con el Pedido, obliga a depurar la diferencia a mano.

Factura — **Noción:** Es el comprobante fiscal que ecoFactory emite a un Cliente por un Pedido, de tipo A, B o C. **Impacto:** Debe corresponderse con el Pedido y con el Remito. No puede modificarse fácilmente una vez emitida, por la integración con AFIP. Puede enviarse al Cliente por mail o por el canal que se elija.

Depósito — **Noción:** Es el lugar físico en la provincia de Buenos Aires donde ecoFactory almacena la Bolsa Ecológica antes de su entrega. **Impacto:** Recibe por mail la indicación de qué Pedido enviar. Genera errores de entrega cuando la comunicación por mail y planillas falla.

Stock — **Noción:** Es la cantidad disponible de Bolsa Ecológica registrada en el Sistema ERP. **Impacto:** Se actualiza mediante altas, bajas y modificaciones registradas por el Operario ERP.

Base de Datos — **Noción:** Es el repositorio de datos del Sistema ERP, no accedido directamente por el personal de ecoFactory. **Impacto:** Es resguardada mediante Backup una vez por día y una vez por semana por un servicio técnico externo.

Backup — **Noción:** Es la copia de resguardo de la Base de Datos del Sistema ERP. **Impacto:** Es realizada por un técnico en informática contratado a terceros, con una frecuencia diaria y semanal.

C.3 Símbolos de tipo Verbo (3)

Facturación — **Noción:** Es la actividad de emitir una Factura a partir de un Pedido, a cargo del Operario ERP. **Impacto:** Se realiza en la práctica de forma manual, entrando al Sistema ERP y cargando cada dato. Genera problemas cuando el Pedido o el Remito no coinciden con lo facturado.

Automatizar — **Noción:** Es la acción que busca el Dueño para reducir el trabajo manual en la gestión de Pedido, Remito y Facturación. **Impacto:** No reemplaza al Sistema ERP; se realiza integrando o comunicando con él. Debe gestionarse con el Responsable ERP cuando implica modificaciones al Sistema ERP.

Integración — **Noción:** Es la acción de conectar el Sistema ERP con software externo mediante una API u otra tecnología estándar de mercado. **Impacto:** Es ofrecida por el Responsable ERP a los clientes del proveedor del Sistema ERP. Ya existe, por ejemplo, con puntos de venta de terceros.

Anexo D — Verificación de ejecución con modelos comerciales vía API

Este anexo deja constancia de una verificación *operativa* realizada sobre el prototipo: comprobar que el pipeline se ejecuta de punta a punta contra la API de un proveedor comercial de LLM, y no solo en la corrida de referencia descrita en el Capítulo 7. Su alcance es el funcionamiento de la herramienta; no constituye una evaluación cuantitativa adicional ni modifica los resultados reportados.

La verificación se realizó configurando en `config.yaml` el proveedor *anthropic* con el modelo *claude-sonnet-4-6* y temperatura 0,2, con la credencial provista por variable de entorno. Se ejecutó la configuración C2c completa —extracción, clasificación, descripción y auto-verificación— tanto desde la línea de comandos como desde la interfaz gráfica, sobre el corpus real de ecoFactory y sobre uno de los casos de muestreo (la clínica veterinaria de la Sección 7.6). En ambos casos las cuatro etapas se completaron y se generaron correctamente el LEL en formato JSON y los dos reportes (el LEL navegable y la evaluación contra el Gold Standard correspondiente a cada corpus).

De la verificación surgieron algunas observaciones prácticas, útiles para quien reproduzca el trabajo. El identificador del modelo debe ser uno vigente y habilitado para la cuenta: los alias genéricos no siempre resuelven. Algunos modelos de última generación *deprecaron el parámetro de temperatura*, por lo que fijar la temperatura —recurso del que depende la reproducibilidad declarada en la Sección 4.13— no es posible con todos ellos; el cliente del prototipo contempla ese caso y continúa la ejecución advirtiéndolo por consola. En la misma línea, la versión 1.x del SDK del proveedor eliminó ese parámetro de su interfaz, de modo que `requirements.txt` acota la dependencia a la línea 0.x. Por último, la etapa de auto-verificación se procesa por lotes de símbolos: al enviarse el LEL completo en una sola solicitud, la respuesta puede exceder el límite de tokens de salida y llegar truncada.

Se deja constancia expresa de que esta verificación no altera lo reportado en el Capítulo 7: la evaluación cuantitativa del trabajo sigue siendo la de la corrida de referencia, con las salvedades allí explicitadas. La corrida ciega sistemática con distintos modelos comerciales, junto con el análisis comparativo de sus resultados, permanece como la línea de trabajo futuro señalada en el Capítulo 9.